

ICT302 IT Professional Practice Project

Project Name:

Secure QR Code Development

Client Organization:

Secure QR Code Project Client

Group Name:

VAFPQR

DESIGN DOCUMENT

Submitted by:

Patricia Orenza [redacted]

Submitted on: 1st Aug 2026

Version: 2.0

Table of Contents

1. Version Control.....5
2. Executive Summary.....6
3. Introduction.....7
3.1 Scope and Purpose.....7
3.2 Intended Audience.....7
3.3 System Identification.....7
3.4 Reference Documents.....8
4. Data Design.....9
4.1 Overview.....9
4.2 Entity relationship design.....10
4.3 Data dictionary.....13
4.4 Data integrity constraints.....22
4.4.1 Entity integrity.....22
4.4.2 Referential Integrity.....22
4.4.3 Domain integrity:.....23
4.4.4 Business constraints.....23
4.4.5 Integrity enforcement mechanisms:.....23
4.5 Data flow overview.....25
4.6 CRUD Matrix.....26
5. Process Design.....27
5.1 Use case list and diagrams.....27
5.2 Process Descriptions and Algorithms.....29
5.2.1 Secure QR Code Generation Algorithm.....30
5.2.2 QR Code Verification Algorithm.....31
5.2.3 Tamper Report Submission Algorithm.....32
5.2.4 Administrator Review and Blacklist Algorithm.....33
5.3 Processing Controls.....34
6. Architecture/Infrastructure Design.....35
6.1 System Architecture.....35
6.1.1 Architectural style.....35
6.1.2 The three layers.....35

6.1.3 End-to-end request flow.....35
6.2. Architecture Diagrams.....36
6.2.1 Layered architecture.....36
6.2.2 User journeys and page navigation.....36
6.3. Additional Software Components.....37
6.4. Infrastructure Requirements.....38
6.4.1 Capacity.....38
6.4.2 Performance.....39
6.4.3 Integration & Compatibility.....39
6.4.4 Platform Strategy.....39
6.4.5 Security.....39
6.4.6 Backup & Recovery.....40
6.4.7 Scalability.....40
6.4.8 Future Proofing.....40
6.5. Alternative Designs Considered.....41
7. Interface Design.....43
7.1 User Interface Design Overview.....43
7.2 Design Foundation.....44
7.2.1 Colour Palette.....44
7.2.2 Typography System.....45
7.3 Mobile Application Interface.....46
7.3.1 Mobile App Welcome Page.....46
7.3.2 Mobile App Onboarding Pages.....46
7.3.3 Mobile App Login.....47
7.3.4 Mobile App Create Account.....47
7.3.5 Mobile App Download Required.....48
7.3.6 Mobile App User Dashboard and Profile.....48
7.3.7 Mobile App QR Scan.....49
7.3.8 Mobile App QR Code Safe and Warning Result.....49
7.3.9 Mobile App User History.....50
7.4 Web Page Interfaces.....51
7.4.1 Admin Login Page.....51
7.4.2 QR Code Display Page (Authenticated Admin).....53

7.4.3 Admin Management Dashboard.....57
7.4.4 Users Management Page.....62
7.4.5 Settings Page.....64
7.5 GUI Components and Input and Output Specifications.....66
8. Appendices.....69
Appendix A: Deliverable Task Breakdown Statement: completed, signed, scanned and copied into document.....69
Appendix B: Glossary/data dictionary.....70
Appendix C: Glossary of terms and definitions and acronyms.....71

1. Version Control

Version	Date	Description of Changes
0.1	20 June 2026	Added introduction and executive summary
0.2	22 June 2026	Added process design, use cases, initial draft of architecture and data design
0.3	24 June 2026	Final architecture and data design
0.4	25 June 2026	Added interface design sections
1.0	27 June 2026	Final changes before submission
1.2	20 July 2026	Changes made based on feedback given
2.0	1 August 2026	Final Submission

2. Executive Summary

This Design Document aims to present the complete technical and architectural design of the development of VAFPQR, a secure QR code system for clients Mr. Peter Cole and Mr. Mike Groeneweg.

Our system will fill a critical gap in the field of security for QR codes, which are used widely and are therefore open to many cyberattacks and can be physically modified. Our platform will provide a multi-tier approach by means of API server, mobile app developed on React Native, three web pages, and PostgreSQL Database.

This document will provide the following information:

1. An introduction to provide the scope, audience, and reference documents for this design.
2. Data Design that details the database schema, entity relationship entity, data dictionary and CRUD matrix.
3. Process Design describing the system's processes through use case narratives, data flow diagrams, and sequence diagrams.
4. Architecture/Infrastructure Design will define the system architecture, component interactions and infrastructure requirements.
5. Interface Design presents the human-computer interface designs for the mobile application, web pages and admin dashboard.
6. The appendices include the Deliverable Task Breakdown Statement, the data dictionary, and the glossary of terms.

The system is to be developed on Node.js with Express as its backend technology, PostgreSQL database with Prisma ORM to store the information, React to build three web pages and React Native to build the mobile application. The JWT based cryptographic signature ensures that it is compatible with the current standard QR readers and fully authenticated for mobile application users.

3. Introduction

3.1 Scope and Purpose

Technical specifications for the development of Secure QR Codes through VAFPQR are presented in detail within this Design Document. Technical design decisions in relation to user interface designs, data structures, processing algorithms, and overall system architecture are derived from the requirements documented in the Requirements and Analysis Document.

All parts of the system including the backend API server, the PostgreSQL database, the three web pages (QR Code Landing Page, QR Code Display Page, and Admin Management Dashboard), and the React Native mobile application are documented in this Design Document which covers the entire scope of the system.

3.2 Intended Audience

The target audience of this document includes all major stakeholders who will take part in implementation of the project. Project manager will use the document for monitoring the design and making sure that it corresponds to the schedule and goals of the project. This document will be useful for developers and team members who will work on designing platform components, considering all necessary criteria listed above. Prior to the beginning of development process, the client will approve the design based on this document. Finally, this document will be used by the supervisor for tracking project's progress and by other maintainers and developers who will continue to develop the system further.

3.3 System Identification

The system is identified as VAFPQR, a Secure QR Code Authentication System, Version 1.0. It consists of the following components:

- Backend API Server – Node.js v20 + Express v4, Version 1.0
- Database – PostgreSQL 15, managed via Prisma ORM v5
- Web/Mobile Pages – Figma (three pages: Landing, Display, Admin Dashboard)
- Mobile Application – React Native 0.73 (Android-first, Version 1.0)

3.4 Reference Documents

This design document is supported by several relevant documents that will aid and guide the project through the design phase.

Document	Description	Relationship
ICT302 Secure QR Code Development – Problem Scope (TM 2026)	Client-provided scope and objectives	Background/ Context
Project Management Plan v1.0 (6 June 2026)	Project scope, schedule, risks, and team structure	Prerequisite
Requirements and Analysis Document (8 June 2026)	Formal functional and non-functional requirements	Prerequisite
Test Plan and Test Results Report	Testing strategy and acceptance criteria	Resulting document
Interactive Prototype	VAFPQR System Development	Resulting document
Final Report	Finalised evaluation of VAFPQR	Resulting document

4. Data Design

4.1 Overview

The secure QR authentication system employs a relational database to store and oversee data that are associated with QR code creation, verification, scan monitoring and incident documentation. PostGreSQL was chosen as the database management system for its dependability, secure features, ability to scale and compatibility with relational data models.

The database is the major storage where all the system activities are stored; it contains QR code data, scanning history, user data, and reports of tampering events. The structure of the system makes possible QR code validation with security and maintaining a full log of the activities and security breaches.

The database is designed based on the normalisation rules. There are primary keys which uniquely identify entries in the entity and there are foreign keys which establish connections between the related entities. With the help of foreign keys, the system can determine which users reported the incidents and which users scanned the QR codes.

Databases consist of four primary entities:

- User
- QrCode
- ScanLog
- Alert
- ActivityLog

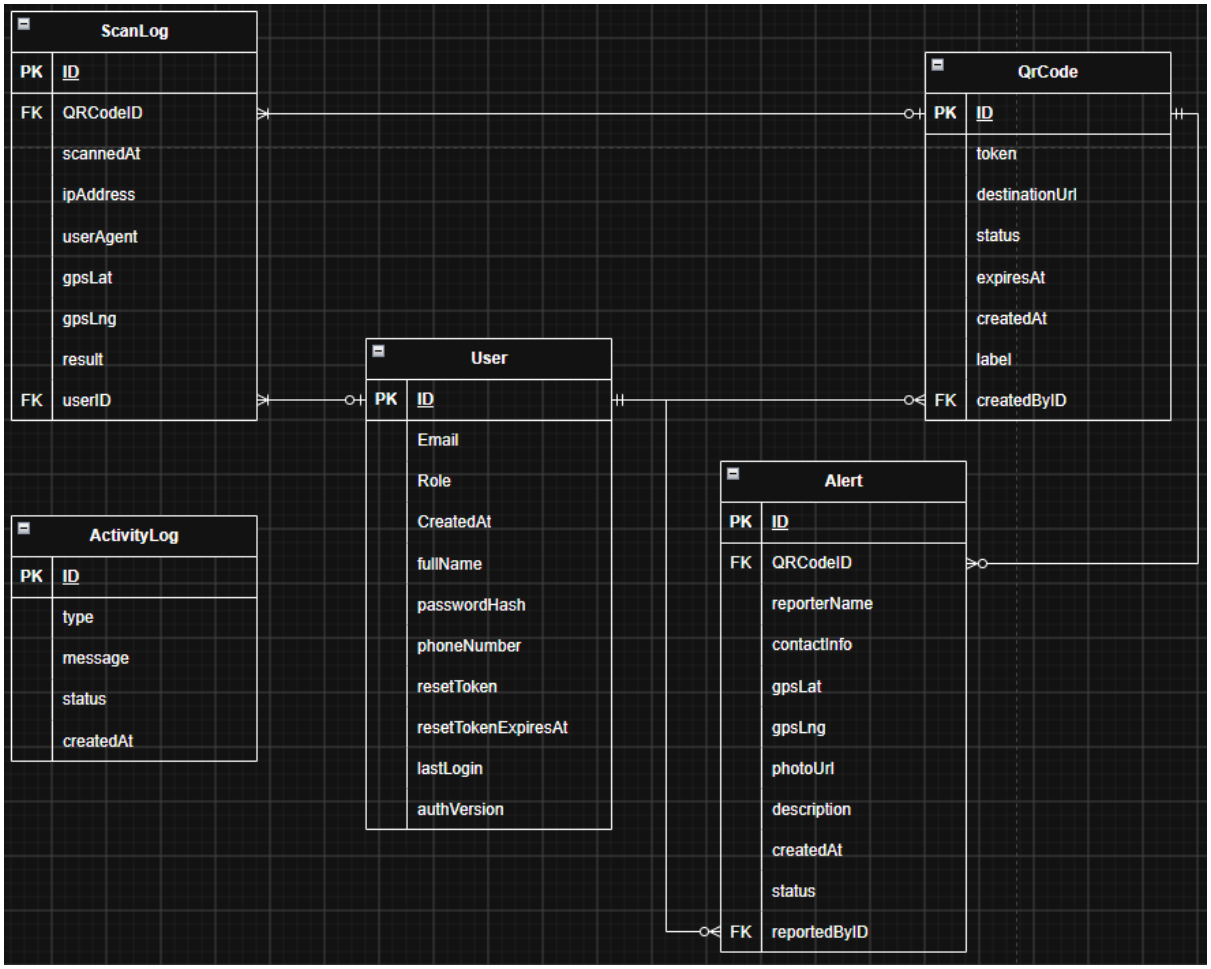
These entities all play a role in supporting all core system functionality including the generation of QR code, authentication, telemetry collection and administrative monitoring.

4.2 Entity relationship design

The Entity Relationship Diagram (ERD) provides the structure of the database along with showing how each of these entities are connected to one another. It has been used in the construction of the database for creating secure QR codes, their scanning, verification, incident reporting, and administration.

Four major entities include User, QR Code, QR Scan Log, and QR Incident Report. All these entities have been connected by foreign keys.

ERD Diagram



This ERD diagram shows the use of five tables: User, QrCode, ScanLog, Alert and ActivityLog.

The User table represents the Users and Administrators of the system. They are differentiated based on their roles given in the ‘Role’ attribute. The ‘resetToken’, ‘resetTokenExpiresAt’ and ‘last Login’ attributes in the table are related to the encryption part of our system. The Primary Key in this table is ID.

The QrCode table represents the QR codes that are tracked within the system. The ‘status’ attribute in the table will assign certain statuses tied to the QR code such as whether they are ‘active’ or ‘suspicious’ etc. The ‘createdAt’ and ‘expiresAt’ attributes show when the QR code was created and when it will expire. The Primary Key in this table is ID.

The ScanLog table represents the individual logs of the scans done through the system. The system will be capable of end-to-end scan tracking from scans done by invalid users to scans done on invalid QR codes by valid users. It will also be able to track the GPS locations of the scans done in the system. The Primary Key in this table is ID.

The Alert table represents the alerts of the QR codes being tampered with sent by users of the system. All the attributes are functions of the table. Examples include ‘gpsLat’ and ‘gpsLng’ showing the GPS location of the scan and ‘photoUrl’ displaying the URL of the photo etc. The Primary Key in this table is ID.

The ActivityLog table represents the recent logs of the activities which will be shown on the admin dashboard. The Primary Key in this table is ID.

Relationships:

User (1) → (0..*) QrCode

An administrator can have zero to multiple QR codes, while a QR code can only be generated by one administrator. This relationship allows the system to identify which administrator generated a specific QR code while also supporting accountability and auditing of QR code management activities.

QrCode (0..1) → (0..*) ScanLog

A QR Code can have zero to multiple scan logs, while a scan log will always refer to a QR code. The only exception is if the QR code is deleted. This supports telemetry collection including the scan time, device information, validation results, location and IP address which can be used for analytics monitoring or security investigations.

QrCode (1) → (0..*) Alert

A QR Code can have zero to multiple tamper alerts, while a tamper alert will always refer to a QR code. This allows the administrator to examine all reports linked to a specific QR code and determine if the QR code should be blacklisted.

User (1) → (0..*) Alert

A user can report zero to multiple tamper alerts, while a tamper alert will always refer to a user. This allows the administrator to trace the reports back to the user who reported it.

User (0..1) → (0..*) ScanLog

A use can scan zero to multiple QR codes while a scan will always refer to a user. The only exception is if the user is deleted. This allows the administrator to trace the user who scanned the QR code.

Summary table

Parent entity	Child entity	Relationship
User	QrCode	1 : 0..N
QrCode	ScanLog	0..1 : 0..N
QrCode	Alert	1 : 0..N
User	Alert	1 : 0..N
User	ScanLog	0..1 : 0..N

4.3 Data dictionary

User: Stores user information that was used to access and manage the secure QR authentication system.

Attribute	Data type	Key	Description	Constraints
Id	TEXT	PK	Unique identifier for user	NOT NULL, UNIQUE
email	TEXT		User’s email address	NOT NULL, UNIQUE
passwordHash	TEXT		Hashed password that will be used for authentication.	NOT NULL
role	TEXT		User role within the system (admin, user).	NOT NULL
createdAt	TIMESTAMP		Date and time when the account was created.	NOT NULL
fullName	TEXT		User’s full name.	NULLABLE
phoneNumber	TEXT		User’s phone number.	NULLABLE
resetToken	TEXT		Password reset token	NULLABLE

resetTokenExpiresAt	TIMESTAMP		Password reset token expiry time.	NULLABLE
lastLogin	TIMESTAMP		Timestamp of when the user’s last login session was.	NULLABLE
authVersion	INT4		Version tracker used for invalidating active tokens.	NOT NULL, DEFAULT 0

QrCode: Stores the secure QR code information and outcome of verification which includes authentication tokens, URL, blacklist status, expiry details and administrator ownership.

Attribute	Data type	Key	Description	Constraints
id	TEXT	PK	Unique identifier for each QR code.	NOT NULL, UNIQUE
token	TEXT		Cryptographically signed JWT token that is used to verify whether the QR code is authentic.	NOT NULL, UNIQUE
destinationUrl	TEXT		URL destination associated with QR code for redirection after successful verification.	NOT NULL
expiresAt	TIMESTAMP		Date and time whereby the QR code becomes invalid.	NOT NULL
createdAt	TIMESTAMP		Date and time whereby the QR code was generated.	NOT NULL
status	TEXT		QR code current operational status (Active, Suspicious, Expired, Blacklisted).	NOT NULL, DEFAULT ‘Active’
label	TEXT		Identifier label for the QR	NULLABLE

			code.	
createdById	TEXT	FK	Reference administrator who generated the QR code (User.id).	NOT NULL

ScanLog: Stores collect scan telemetry data and the verification results to assist in monitoring, auditing, analysis and security inquiries.

Attribute	Data type	Key	Description	Constraints
id	TEXT	PK	Unique identifier for each scan record.	NOT NULL, UNIQUE
scannedAt	TIMESTAMP		Date and time of when the QR code was scanned.	NOT NULL
ipAddress	TEXT		IP address from where the scan request is requested.	NULLABLE
userAgent	TEXT		Browser and device user agent details.	NULLABLE
gpsLat	FLOAT8		Scan location’s latitude coordinate.	NULLABLE
gpsLng	FLOAT8		Scan location’s longitude coordinate.	NULLABLE
result	TEXT		Verification result (Valid, Invalid, Expired, Blacklisted)	NOT NULL
qrCodeId	TEXT	FK	Reference	NOT NULL

			the QR code scanned (QrCode.id).	
userId	TEXT	FK	References to the user who performed the scan if needed (User.id).	NULLABLE

Alert: Keeps a record of alleged QR code tampering along with the proof and location information for administrator investigation.

Attribute	Data type	Key	Description	Constraints
id	TEXT	PK	Unique identifier for each incident report.	NOT NULL, UNIQUE
reporterName	TEXT		Name that was provided by the user who submitted the report.	NULLABLE
contactInfo	TEXT		Contact information optionally provided by the reporter.	NULLABLE
gpsLat	FLOAT8		Latitude coordinates whereby the alleged tampering was found.	NOT NULL
gpsLng	FLOAT8		Longitude coordinates whereby the alleged tampering was observed.	NOT NULL
photoUrl	TEXT		URL path or storage link that points to the uploaded photographic evidence.	NOT NULL
description	TEXT		Explanation provided by user	NOT NULL

			regarding the alleged QR code tampering.	
createdAt	TIMESTAMP		Date and time whereby the alert was submitted.	NOT NULL
status	TEXT		Status of the incident report currently (Pending, Resolved, Under investigation) .	NOT NULL, DEFAULT ‘Pending’
qrCodeId	TEXT	FK	Reference the QR code associating with the report (QrCode.id).	NOT NULL
reportedById	TEXT	FK	Reference to the user who submitted the tampering report.	NOT NULL

AcitivityLog: Track system-level and administrative actions for auditing reasons.

Attribute	Data type	Key	Description	Constraints
id	TEXT	PK	Unique identifier for the activity log entry.	NOT NULL, UNIQUE
type	TEXT		Type or category of the activity logged.	NOT NULL
message	TEXT		Detailed description of the occurrence or activity.	NOT NULL
status	TEXT		Status or outcome of the activity.	NULLABLE
createdAt	TIMESTAMP		Timestamp of when the activity happened.	NOT NULL
qrCodeId	TEXT	FK	Optional reference to the specific affected QR code (QrCode.id),	NULLABLE

4.4 Data integrity constraints

The database enforces multiple integrity constraints to ensure reliability and consistency. These constraints play a part in preventing any entry of invalid data and maintaining relationships between database entities.

4.4.1 Entity integrity

All entities contain primary keys that uniquely identify records. This ensures that each administrator, scan record, QR code and incident report can be uniquely identifiable within the system.

Entity	**Primary Key**
User	id
QrCode	id
ScanLog	id
Alert	id
ActivityLog	id

4.4.2 Referential Integrity

Foreign key relationships to ensure valid connections between entities. This ensures that all scan logs and incident reports are linked to a valid QR code and that each code is linked to a valid administrator.

Child entity	Parent entity	Foreign Key	Primary Key
QrCode	User	createdById	id
ScanLog	QrCode	qrCodeId	id
ScanLog	User	userId	id
Alert	User	reportedById	id
Alert	QrCode	qrCodeId	id

4.4.3 Domain integrity:

Guarantees that attribute values adhere to their specified formats, data types and permissible values.

- UUID attributes need to have valid UUID values.
- URLs saved in the QrCode entity should adhere to proper URL formatting.
- Date and time fields must include valid timestamp values.
- Validation outcome result values should solely consist of established verification results.
- Details input should not surpass the specified character limit.

4.4.4 Business constraints

Following business rules applied:

Each QR code must possess a unique id.

- Each QR code must include a legitimate URL destination.
- Each QR code is required to include a legitimate authentication token.
- JWT tokens need to be created by the server and associated with a particular QR code.
- Expiration date should come after the QR code generation date.
- Newly created QR codes should have a default status set to Active.
- QR codes that are marked as suspicious must display a warning message.
- QR codes that are blacklisted should not redirect user to the designated URL.
- Logs cannot exist without a corresponding QR code record.
- Incident reports must include a description of the problem.
- Each evidence photo uploaded must be associated with a valid report.
- Passwords must be stored in hashed form and never kept as a plain text.
- Each QR code must be linked to a legitimate administrator account
- Results of QR code verification should be documented in the ScanLog table for auditing and oversight reasons.
- Contact number if provided should only contain valid phone numbers.

4.4.5 Integrity enforcement mechanisms:

The secure QR authentication system maintains database integrity through a combination application level and database level controls.

Database level controls:

- Primary key constraints
- Foreign key constraints
- NOT NULL constraints
- UNIQUE constraints
- Data type validation
- Role Level Security (RLS) policies

RLS: The database enforces RLS policies that we wrote to limit row-level access according to user authentication and roles. For example, regular users can safely

send incident reports (Alert) and records scans (ScanLog) whereas administrative accounts have elevated privileges to handle QR codes (QrCode), examine all the logs and modify any alert statuses across the system.

Application-level controls:

- JWT token signature verification
- URL format validation
- QR code expiry validation
- QR status validation
- Authentication and authorization checks
- Input validation and sanitisation

4.5 Data flow overview

The data is gathered through the QR code generation process which consists of scanning QR codes and submission of incident reports. Each time an administrator generates QR codes, the system generates a unique QR id and a cryptographic token embedded in the designated URL. The QrCode table holds the information about the QR code including destination URL, status, expiry date and information regarding the generator of the QR code (createdById).

When a QR code is scanned, the system validates the JWT token, checks the expiry date of the QR code and examines the QR Status. Depending on the outcome of the validation, the system shows either a warning message to the user or redirects the user to the intended destination URL. The ScanLog table logs the scanning activity along with the timestamp (scannedAt), IP Address, validation outcome (result), location and device type.

If a user spots any suspicious QR code, they can file an incident report through the mobile application. The Alert table contains the report along with the photographic evidence, GPS location, contact information if provided, description and the status of the report. The QR code that is linked might be given a suspicious status while awaiting administrator evaluation. The information gathered and provided plays a huge role in security evaluation, system oversight, reporting and administrative choices through the admin management dashboard.

4.6 CRUD Matrix

The CRUD Matrix specifies the processes that can Create (C), Read (R), Update (U) or Delete (D) data within each entity. It illustrates the interaction of each system process with the underlying database entities while offering traceability between the data stored and business functions.

Process	User	QrCode	ScanLog	Alert	ActivityLog
User login	R				
User registration	C				
Manage user profile	U				
Generate QR code		C			
Verify QR code		R	C		
Submit incident report	R	U		C	R
View dashboard	R	R	R	R	R
Manage QR codes		U	R	R	

Legend
C = Create
R = Read
U = Update
D = Delete

5. Process Design

VAFPQR System involves three distinct user types that will be interacting with the platform.

1. Regular Users: members of the public wo scan QR codes using their device's built-in camera app which leads to a redirection to download the VAFPQR application.

2. App Users: individuals who have installed the Secure QR mobile application and have created an approved account. They receive fill JWT validation and can submit tamper alerts.

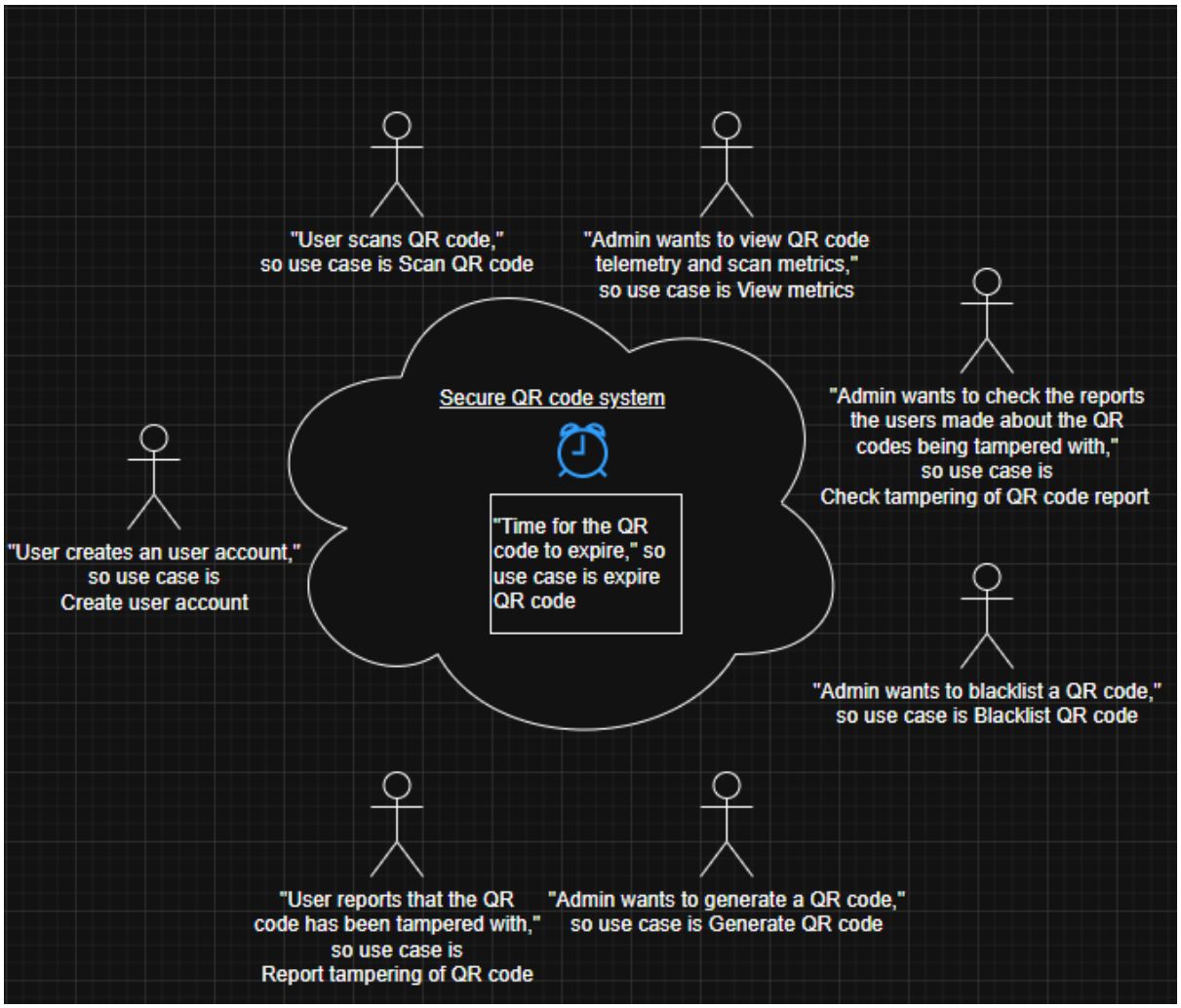
3. Admin Users: authenticated admins who access the web-based Admin Management Dashboard

5.1 Use case list and diagrams

Use case list

Secure QR code System	
Use cases	Users/actors
Scan QR code	User
Create user account	User
Report tampering of QR code	User
View metrics	Admin
Check tampering of QR code report	Admin
Blacklist QR code	Admin
Generate QR code	Admin

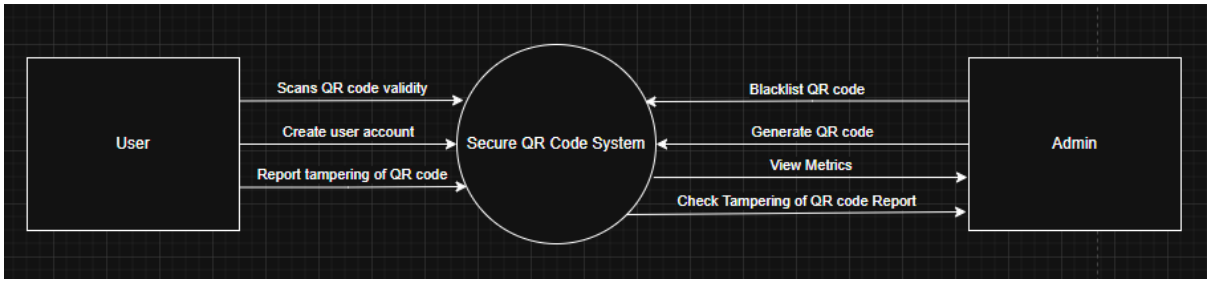
Event decomposition diagram



The QR code automatically expires once the expiry time of the QR code is reached. Which makes it a temporal event.

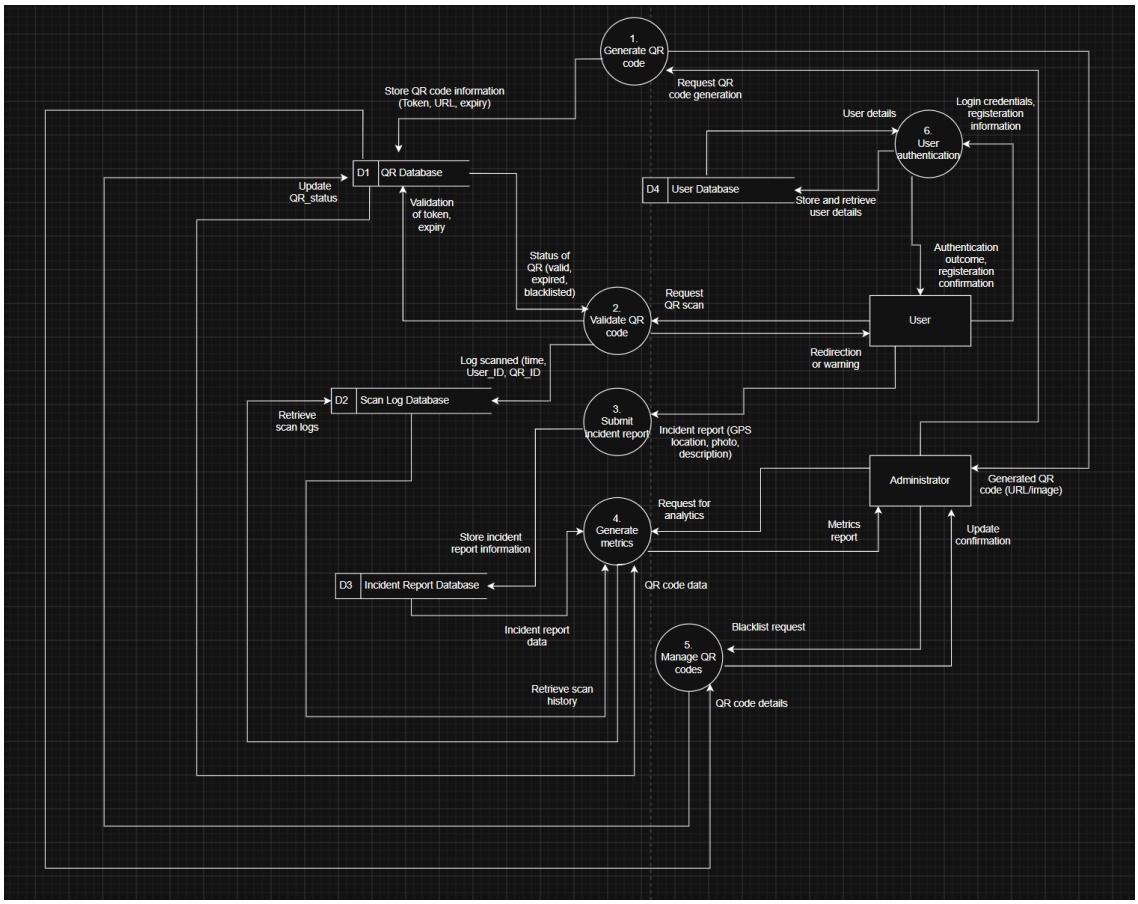
The other events outside of the system such as “Scan QR code” or “View metrics” to name a few, are initiated by the User and Admin. These events are called external events.

Level 0 DFD



The user interacts with the Secure QR Code System to scan the validity of the QR code, create user accounts or report the QR code being tampered with.
The admin interacts with the Secure QR Code System to blacklist or generate QR codes, retrieve metrics of the QR codes and check the QR Code tampering reports.

Level 1 DFD



The Administrator can request a QR code to be generated by the QR code generator by inputting the redirected URL and amount of time before expiry. The QR code will then be stored in the QR Database.
The user can validate the QR code by scanning it after which the QR code will be cross-referenced with the QR Database to check for its validity. Its result will then be shown to the user. Any relevant details of the scans of the QR code will be stored in the Scan Log Database.
The user can submit an incident report if the QR code has been tampered with. It will then be stored in the Incident Report Database which the Administrator can check to verify the report's validity. The Administrator can then blacklist the QR code in the QR Database.
The Administrator can retrieve the scan telemetry and metrics which are stored in the Scan Log Database.

5.2 Process Descriptions and Algorithms

This section describes the main processing logic of the Secure QR Code Authentication System. The algorithms are written in structured English to show how each main function should be processed by the system before implementation. The

purpose of this section is to support the development of backend API, mobile application, web pages and database operations.

5.2.1 Secure QR Code Generation Algorithm

- Purpose:

To allow an administrator to generate a secure QR code from the QR Code Management Page by entering a destination URL and selecting an expiry period.

- Input:

Administrator ID, destination URL, selected expiry hours.

- Output:

Generated QR code image, QR code details, and stored QR code record.

- Process:

1. The administrator opens the QR Code Management Page.
2. The administrator enters the destination URL in the Destination URL input field.
3. The administrator selects an expiry period from the Expiry Hours dropdown. The available options are 1, 48, 72, and 168 hours.
4. The administrator clicks the “Generate QR Code” button.
5. The system checks whether the current user has administrator permission.
6. The system validates that the destination URL is not empty and follows a valid URL format.
7. The system checks whether the selected expiry hours match one of the allowed expiry options.
8. If the destination URL or expiry hours are invalid, the system displays an error message and stops the process.
9. If the input is valid, the system generates a unique QR.id.
10. The system calculates the expiry date and time based on the selected expiry hours.
11. The system creates a signed JWT token using the QR.id, destination URL, issue time, and expiry time.
12. The system stores the QR code details in the QrCode table, including the QR.id, destination URL, signed token, expiry date, created time, QR status, and administrator ID.
13. The QR status is set to Active when the QR code is first generated.
14. The system generates a QR code image that contains the verification link and signed token.
15. The system displays the generated QR code image and QR code details on the result display area.
16. The administrator can then download, print, or view the generated QR code details.

5.2.2 QR Code Verification Algorithm

- Purpose:

To verify whether a scanned QR code is valid, active, and safe before allowing the user to access the destination URL.

- Input:

Scanned QR token, device information, IP address, and optional location data.

- Output:

Verification results, scan log records, and redirect decisions.

- Process:

1. The user scans the QR code using either a normal QR scanner or a dedicated mobile application.
2. The system receives the signed token through the QR code verification endpoint.
3. The system checks whether the token is included in the request.
4. If the token is missing, the system records the scan as invalid and displays an invalid QR code message.
5. If the token is included, the system verifies the JWT signature.
6. If the JWT signature is invalid, the system records the scan as invalid and displays a warning message.
7. If the JWT signature is valid, the system checks whether the token has expired.
8. If the token has expired, the system records the scan as expired and displays an expired QR code message.
9. If the token has not expired, the system searches the QrCode table using the QR.id inside the token.
10. If the QR code record cannot be found, the system records the scan as invalid and displays a warning message.
11. If the QR code record is found, the system checks the status value.
12. If the status is Blacklisted, the system blocks the redirect and displays an unsafe QR code warning.
13. If the status is Suspicious, the system displays a warning message and prevents automatic redirection.
14. The system ddetermines the validation outcome based on the token result, expiry check, database record, and status.
15. The system stores the scan time, validation outcome, IP address, location if available, device type, and QR.id in the ScanLog table.
16. The system checks the user-agent to identify whether the scan came from the dedicated mobile application or from a normal browser scanner.
17. If the scan came from the dedicated mobile application, the system returns the verification result to the mobile app.

18. If the scan came from a normal browser scanner, the system displays the QR Code Landing Page with the verification result and destination URL.
19. The user is only allowed to continue to the destination URL if the QR code is valid and active.

5.2.3 Tamper Report Submission Algorithm

- Purpose:

To allow a mobile application user to report a suspicious or physically tampered QR code with supporting information.

- Input:

QR.id, report description, GPS location, photo evidence, and optional contact details

- Output:

Stored tamper report record and updated QR code status

- Process:

1. The user scans a QR code using the dedicated mobile application.
2. If the user notices that the QR code may be suspicious or physically tampered with, the user selects the report option in the mobile application.
3. The system retrieves the QR id from the scanned QR code.
4. The user enters a description of the issue in the report form.
5. The user is required to attach a photo as evidence.
6. The system collects GPS location if the user gives location permission.
7. The user may provide optional contact details if further follow-up is needed.
8. The system checks whether the QR.id exists in the QrCode table.
9. If the QR.id does not exist, the system displays an error message and stops the report submission process.
10. If the QR id exists, the system creates a new tamper report record in the Alert table.
11. The system stores the QR.id, report description, GPS location, photo evidence, report time, and optional contact details.
12. The system updates the related QR code status to Suspicious.
13. The system displays a confirmation message to inform the user that the report has been submitted successfully.
14. The report becomes available in the Admin Dashboard for administrator review.

5.2.4 Administrator Review and Blacklist Algorithm

- Purpose:

To allow an administrator to review submitted tamper reports and update the status of a suspicious or compromised QR code.

- Input:

Selected tamper report, QR code details, scan history, and administrator decision

- Output:

Updated QR code status and reviewed report information

- Process:

1. The administrator opens the Admin Dashboard.
2. The system displays the list of tamper reports submitted by mobile application users.
3. The administrator selects a tamper report for review.
4. The system displays the related QR code details, report description, report time, GPS location if available, photo evidence if available, and scan history.
5. The administrator reviews the report information and checks whether the QR code may have been physically replaced or tampered with.
6. If the report does not appear to be valid, the administrator may leave the QR code status as Active or keep it under review.
7. If the report appears to be valid or serious, the administrator updates the QR code status to Blacklisted.
8. The system updates the status value in the QrCode table.
9. If the QR code is marked as Blacklisted, future scans of the same QR code will be blocked by the verification process.
10. The system keeps the report record for audit and future reference.

5.3 Processing Controls

Processing controls are incorporated into VAFPQR in order to guarantee the proper execution of the principal system processes. The processing controls will be primarily used in the backend API as the backend is responsible for validation of the QR code, token validation, scanning recording and restriction of the access to the destination URL.

In the QR Code Management Page the administrator should input the destination URL and set the expiry hours of the QR code prior to generating the QR code. The inputs need to be verified prior to creation of the QR code records in the backend to prevent the generation of the invalid QR codes due to the lack of a URL or expiry field or invalid URL format. The request for generation should be rejected, and the QR code generation should be prevented.

If the QR code gets scanned, then the QR code validation step must verify the signed JWT token before letting the user continue. This means the validation of token signature and token expiration. The user should not be redirected to the destination URL if there is no token, token got changed or the token is expired.

The backend also needs to verify the result of scanning with the QrCode table. Although the token can be successfully decoded, the status still needs to be verified as there might be the case when the QR code gets marked by the administrator as Suspicious, Expired or Blacklisted due to its vulnerabilities. Therefore, the redirect cannot happen in a regular manner.

Any scan activities carried out through the system are recorded in the ScanLog table. This will include the scan time, validation result, IP address, device type, possible location data and corresponding QR.id.

Management operations can only be accessed by administrative users. Only the Admin Dashboard will enable the administrators to create QR codes, track scan activity, report on tampering and update QR code status. Ordinary mobile application users or normal QR scanners should not have access to changing QR code entries and security statuses.

In case the backend API cannot make some critical operation such as token authentication or database query, it would be safer to stop the redirect process. This would avoid any cases where users could be redirected to an unverified destination URL.

6. Architecture/Infrastructure Design

6.1 System Architecture

6.1.1 Architectural style

The system uses a three-tier, layered client–server architecture. The three tiers are a presentation layer (the things users interact with), an application layer (a single Express API that holds all the logic), and a data layer (the database and file storage). Each tier only talks to the one directly below it, which keeps the design easy to test, secure, and extend.

Core design principle — all security logic lives on the server. The printed QR code only encodes a plain HTTPS verification URL. Nothing secret or executable is stored in the code itself. This single decision is what makes the system backward-compatible with the ISO/IEC 18004 standard: any built-in Apple or Google camera can read the code, while the signature checking, expiry, blacklist, and routing all happen server-side.

6.1.2 The three layers

- Presentation layer — regular QR scanners (native phone cameras), the React Native mobile app, and three React web pages: the QR Landing Page, the QR Display Page, and the Admin Management Dashboard.
- Application layer — a Node.js + Express REST API. Four endpoint groups (generate, verify, alert, admin) sit on top of three shared services: a JWT token service, user-agent detection, and a multer file-upload handler.
- Data layer — PostgreSQL accessed through the Prisma ORM (tables: QrCode, User, ScanLog, Alert, ActivityLog), plus cloud object storage for alert photos. The database stores only the photo URL, not the image bytes.

6.1.3 End-to-end request flow

1. An admin generates a code (POST /api/qr/generate): the server signs a JWT, builds a QR image encoding the verify URL, and saves the record with status = active. The QR Display Page is shown.
2. A user scans the code with any scanner. The request reaches GET /api/qr/verify?token=…
3. The server validates the signature and expiry, checks the blacklist/whitelist, detects the user agent, and logs the scan to ScanLog.
4. App users (User-Agent SecureQRApp/1.0) get an in-app result; browser users are served the Landing Page showing a safe/warning result and an app-download prompt.
5. If tampering is suspected, a report is sent to POST /api/alert/report. Multer stores the photo in cloud storage, the GPS and description go to the alerts table, and the code's status is set to suspicious.
6. Admins monitor everything through the dashboard: metrics, QR management, and alert review.

6.2. Architecture Diagrams

6.2.1 Layered architecture

The diagram below maps every component to its tier and shows how a client request travels down to the data layer. All client-to-server traffic is encrypted with HTTPS/TLS.

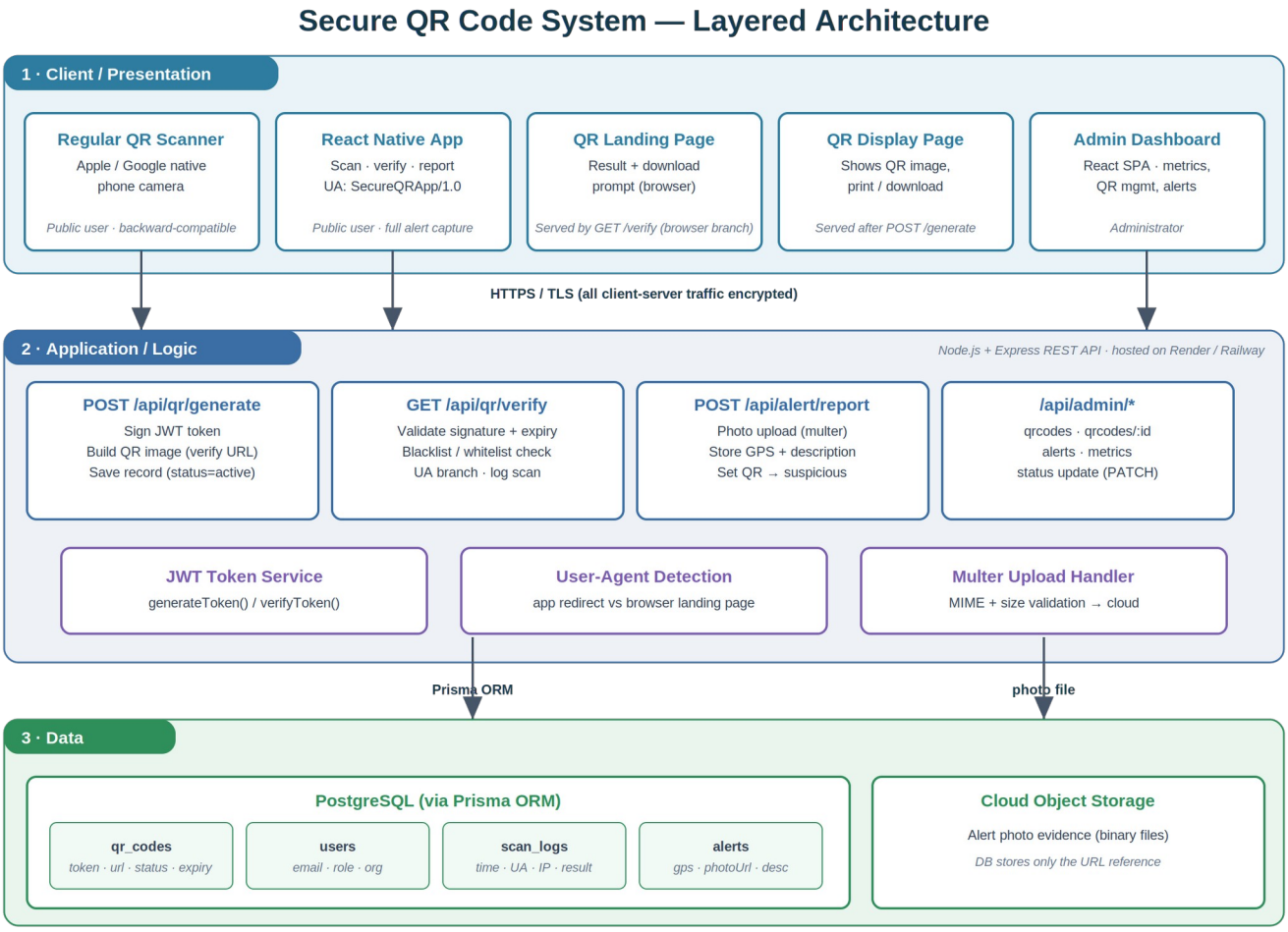


Figure 1 — Layered system architecture

6.2.2 User journeys and page navigation

This diagram shows the two main journeys: the public scan flow (top), which branches by user-agent into the app or the browser Landing Page, and the admin dashboard navigation (bottom), including the generate-to-display path.

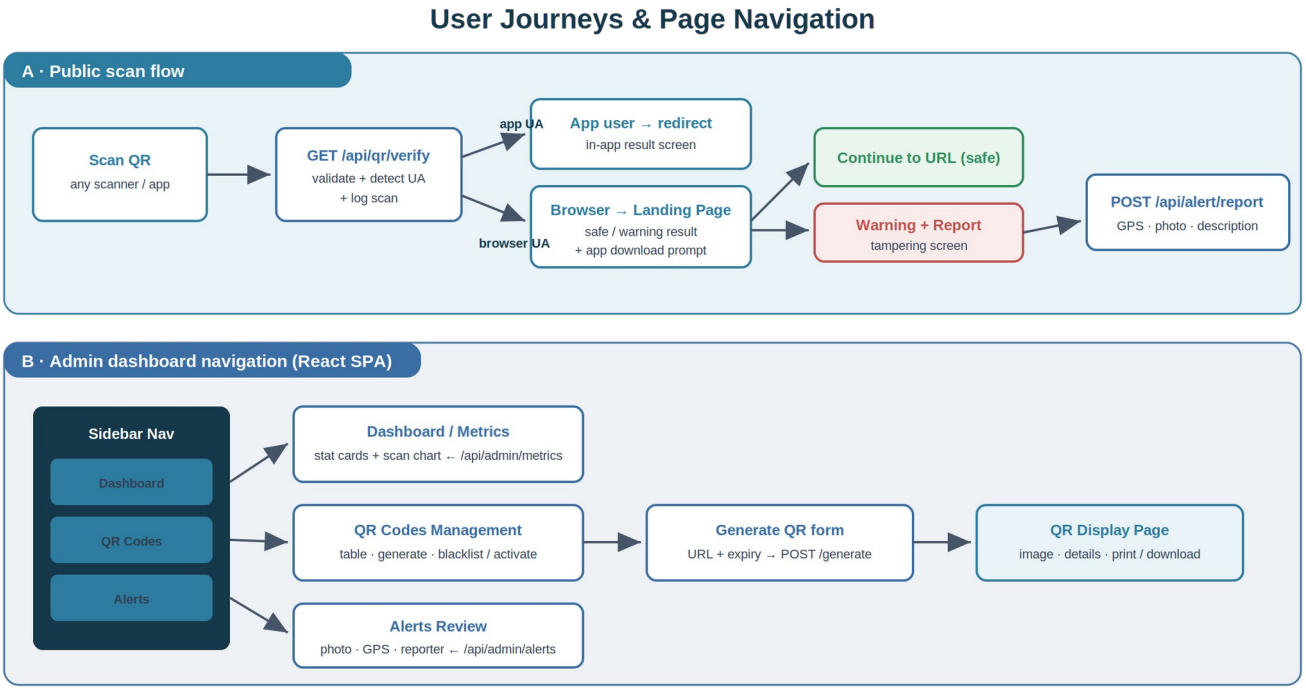


Figure 2 — User journeys and web page navigation

6.3. Additional Software Components

Beyond the core stack (Node.js, Express, PostgreSQL, React, React Native), the system relies on the following libraries and services:

Component	Purpose
jsonwebtoken	Signs and verifies the JWT tokens that make each QR code tamper-evident.
qrcode	Generates the QR code image (encoding the verify URL) as a base64 PNG.
multer	Handles photo-evidence file uploads on the alert endpoint, with MIME and size validation.
Prisma ORM	Type-safe database access and schema migrations against PostgreSQL.
axios	HTTP client used by the dashboard and mobile app to call the API.
react-router-dom	Client-side routing / navigation in the Admin Dashboard

	SPA.
recharts	Renders the scan-metrics charts on the dashboard.
Expo Camera / react-native-camera	Camera access for QR scanning and in-app photo capture on mobile.
helmet & cors	Express middleware for secure HTTP headers and controlled cross-origin access.
dotenv	Loads secrets (DATABASE_URL, JWT secret) from .env so they stay out of source control.
Cloud object storage	Stores alert photos; the database keeps only the returned URL.
Render / Railway	Managed hosting for the Express backend and PostgreSQL database.

6.4. Infrastructure Requirements

6.4.1 Capacity

ScanLog is the fastest-growing table — the design assumes far more scans than codes or alerts. Every QR code is one row; every scan adds a row to ScanLog; only suspected tampering adds an alert. For this academic phase the expected volumes are modest, but the schema separates logs and alerts so the busy write path stays isolated.

Dimension	Expected volume (this phase)	Notes
QR codes	Tens to low hundreds	One record each; small footprint.
Scan transactions	Low thousands (demo/test)	Highest-frequency operation; main write load.
Alerts + photos	Few	Photos in cloud storage, URL only in DB.

Admin users	A handful	Dashboard access only.
Public scanners	Potentially many	Stateless verify scales with load.

City-level GPS is recorded for telemetry rather than precise coordinates, which limits data volume and reduces privacy risk.

6.4.2 Performance

The verify/redirect path must complete in 3 seconds or less. This is the system's headline performance target because it sits in the user's scan-to-result path. QR generation and scan success are both targeted at 99% or higher.

- Stateless JWT verification avoids a database session lookup, keeping verify fast.
- An index on the token column speeds the verify and admin lookups.
- QR images are generated once at creation time, not on every scan.
-

6.4.3 Integration & Compatibility

Backward compatibility with standard ISO/IEC 18004 scanners is the hardest constraint, and it shapes the whole design. Because the code only carries a plain HTTPS URL, any native camera can scan it without special software.

- User-agent detection (SecureQRApp/1.0) lets the same endpoint serve app users and browser users differently.
- All communication is over HTTPS.
- No integration with external enterprise systems is in scope for this phase.

6.4.4 Platform Strategy

A single, cross-platform JavaScript stack is used end to end. Node.js + Express runs the backend on a Linux host (Render or Railway); PostgreSQL is a managed cloud database; React powers the web pages and React Native powers the mobile app (Android-first). Development is on Windows, with GitHub for version control.

Rationale: one language across the whole stack lowers the learning curve for the team, and every tool has a free tier suitable for an academic project.

6.4.5 Security

Tokens are tamper-evident and compromised codes are blocked server-side. Each QR code carries a signed JWT, so any altered token fails signature validation. Codes that are reported or known-bad are blacklisted and refuse to redirect.

- HTTPS/TLS protects all traffic; certificate renewal is tracked to avoid the expiry risk (R12).
- Role-based access restricts the admin dashboard and admin endpoints.

- Uploads are restricted to images and size-limited, with MIME-type validation, to block malicious files (R14).
- Secrets (database URL, JWT secret) live in .env and are never committed to GitHub.
- City-level GPS and storing only photo URLs reduce the amount of sensitive data held.

6.4.6 Backup & Recovery

Automated daily database backups are paired with a defined disaster-recovery procedure. This directly addresses the data-loss and downtime risks (R06, R07) in the risk register.

- Database replication provides an extra copy of QR, scan, and alert data.
- Recovery steps cover service restart and database restoration from the latest backup.
- Photo evidence inherits the redundancy of the cloud storage provider.
- Monitoring raises an alert if downtime exceeds three minutes.

6.4.7 Scalability

A stateless backend allows the system to scale horizontally. Because authentication is JWT-based with no server-side session, multiple identical API instances can run behind a load balancer without sharing state.

- The managed PostgreSQL tier can be scaled up as data grows.
- Offloading photos to cloud storage keeps the database small and fast.
- Separate ScanLog and alerts tables let the high-write log path grow independently.

6.4.8 Future Proofing

The design favours a modular, mainstream, well-supported stack over leading-edge technology. Layers are loosely coupled, so a component (hosting, storage provider, even the database) can be swapped with limited impact. This is a deliberate balance: a modest, proven implementation is more maintainable within the project timeline than a cutting-edge one.

- A Progressive Web App path can later satisfy the client's preference for no mandatory app on the public side.
- The optional extras (an ISO 18004 extension and a time-based redirect service) can be added on top of the existing endpoints.
- Scan-location data already collected enables future anomaly detection for replication attacks (R04).

6.5. Alternative Designs Considered

Several alternatives were weighed before settling on the current design. The table summarises each decision and why the chosen option won.

Decision area	Chosen approach	Alternative considered	Why the chosen approach
QR payload	Plain HTTPS redirect URL; logic server-side	Encrypt / hash data inside the QR payload	Embedding ciphertext breaks standard scanners and fails the backward-compatibility requirement.
Mobile delivery	App plus a browser / PWA landing path	Native-app-only	The client preferred not to force an app; a browser path widens adoption (R10).
Backend shape	Single monolithic Express service	Microservices	Microservices add operational overhead not justified at this scale or timeline.
Database	Relational PostgreSQL	NoSQL / document store	Clear relationships (code → scans → alerts) and metric joins suit a relational model.
Photo storage	Cloud object storage, URL in DB	Store images as BLOBs in the database	Keeps the database small and fast, is cheaper, and is easier to back up and scale.
Authentication	Stateless JWT	Server-side sessions	Stateless tokens scale horizontally and are self-validating, with no session store.

7. Interface Design

7.1 User Interface Design Overview

The objective of the design of the interface is to develop a simple, friendly, and secure interface for mobile users and administrators. The design focuses on being simple, easy to use, and facilitating the secure transfer of validation messages of QR codes.

A design system is consistently used throughout all websites and the mobile app.

Design principles:

- Consistency: Using common elements and interactions in all interfaces
- Feedback: Indication of validation messages and system status
- Simplicity: Minimal actions from the user necessary for basic functions
- Accessibility: Accommodates users with various technical skills
- Responsiveness: Interfaces work on various devices

Interfaces are designed using iteration and user feedback. Design of interfaces is done in accordance with the requirements outlined in the Requirements and Analysis Document and Project Management Plan.

7.2 Design Foundation

The following colour palette is specifically tailored to the web-based interfaces of the VAFPQR application, which includes the Admin Login Page, QR Code Display Page, Admin Management Dashboard, Users Management Page, and the Settings Page. The mobile application (see Section 7.3) follows platform-specific design standards while ensuring visual consistency when necessary.

7.2.1 Colour Palette

Colour Name	Hex Code	Colour Swatch	Usage
Primary Blue	#1A56DB		Buttons, headers, active states
Primary Light	#3B82F6		Hover states, secondary elements
Success Green	#059669		Active QR codes, success messages
Danger Red	#DC2626		Blacklisted QR codes, errors
Warning Yellow	#F59E0B		Suspicious QR codes, warnings
Dark Slate	#1F2937		Text, headings
Medium Gray	#6B7280		Secondary text, icons
Light Gray	#F3F4F6		Backgrounds, cards
White	#FFFFFF		Content backgrounds

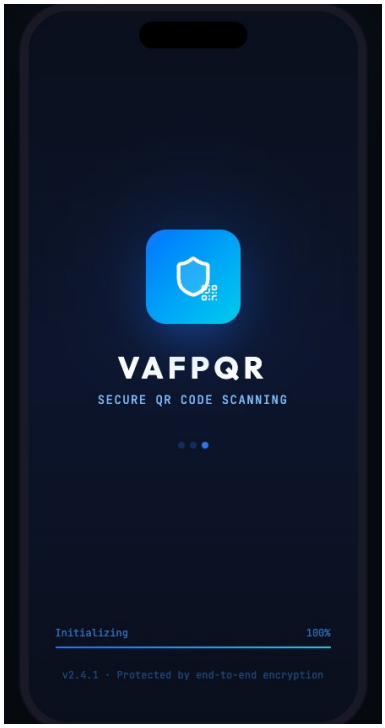
7.2.2 Typography System

Style	Size	Weight	Line Height	Usage
Heading 1	32 px	Bold (700)	40 px	Page titles, screen names
Heading 2	24 px	Semibold (600)	32 px	Section headings, card titles, dashboard widget labels
Heading 3	20 px	Semibold (600)	28 px	Sub-section labels, table column headers, form group labels
Body	16 px	Regular (400)	24 px	Paragraphs, table cell content, form field input text
Body Small	14 px	Regular (400)	20 px	Helper text, placeholder text, metadata (timestamps, scan counts)
Caption	12 px	Regular (400)	16 px	Status badge labels, image captions, GPS coordinate display

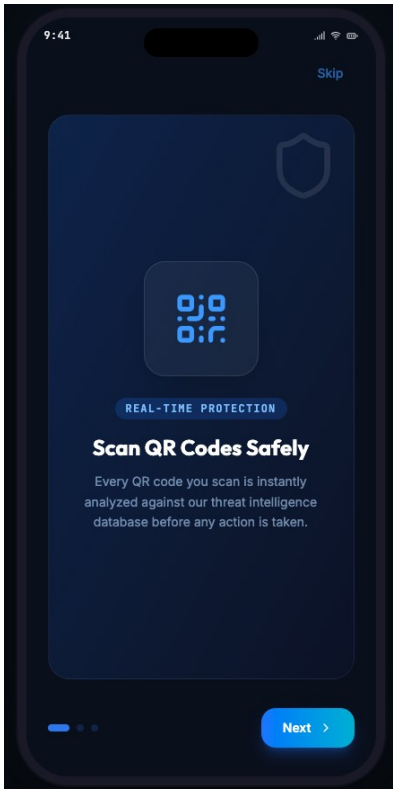
7.3 Mobile Application Interface

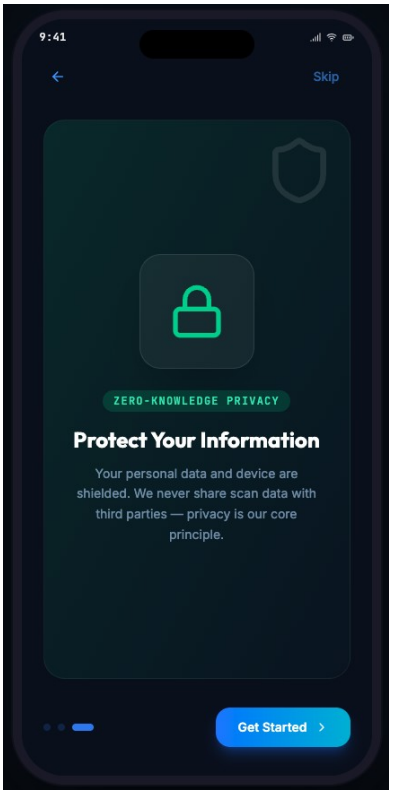
VAFPQR Mobile Application consists of a welcome page → onboarding page → login/register account → scan QR code → QR code result → summary/reporting page.

7.3.1 Mobile App Welcome Page



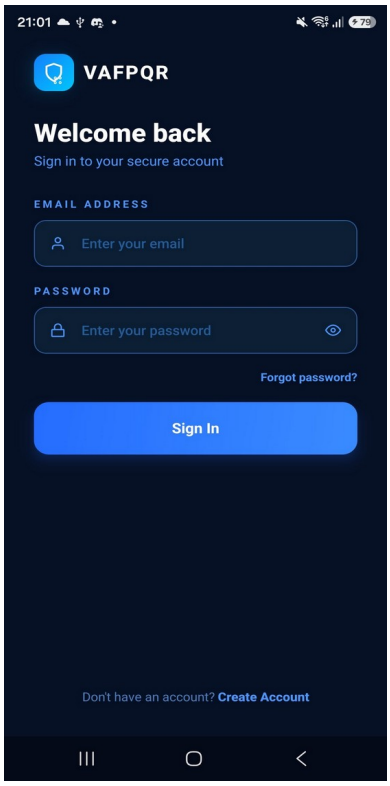
7.3.2 Mobile App Onboarding Pages



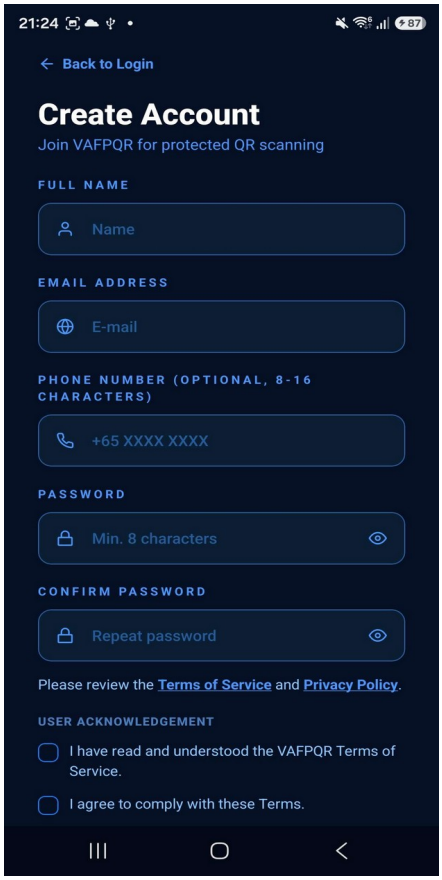


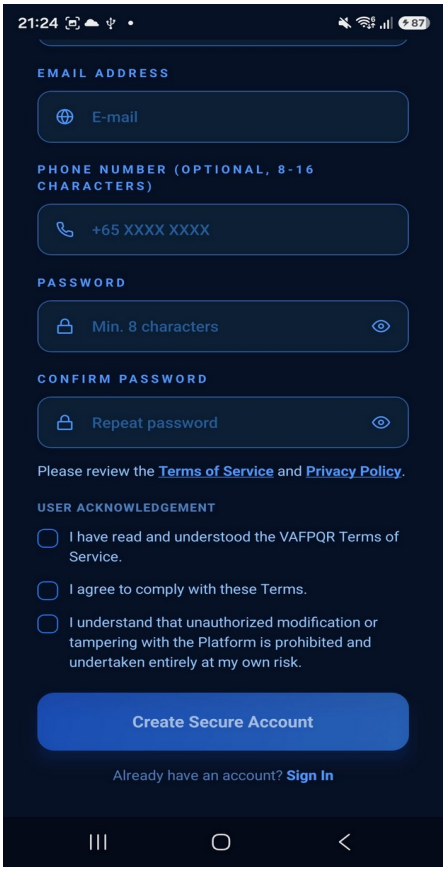


7.3.3 Mobile App Login

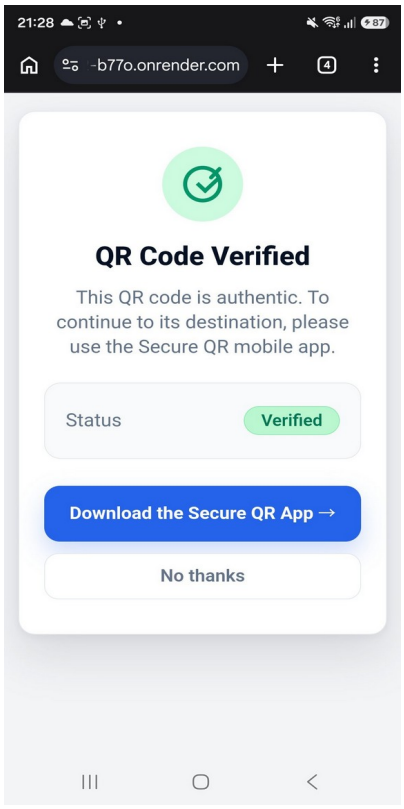


7.3.4 Mobile App Create Account

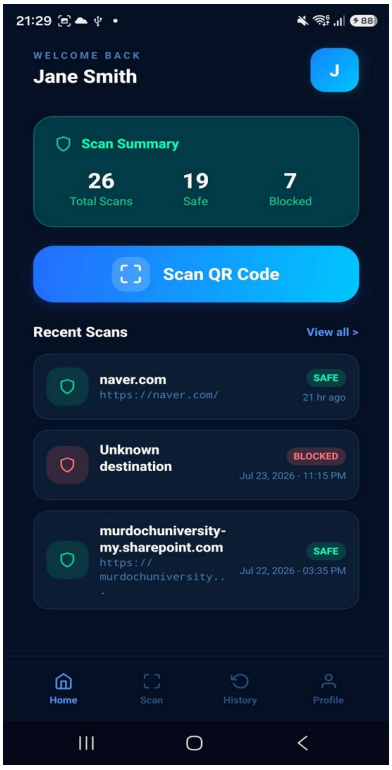


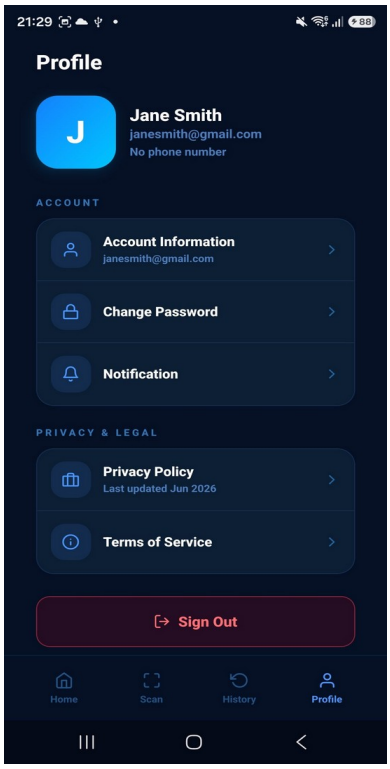


7.3.5 Mobile App Download Required

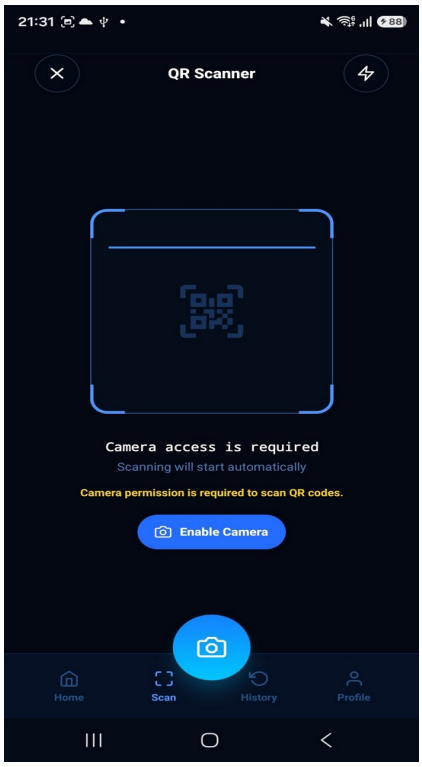


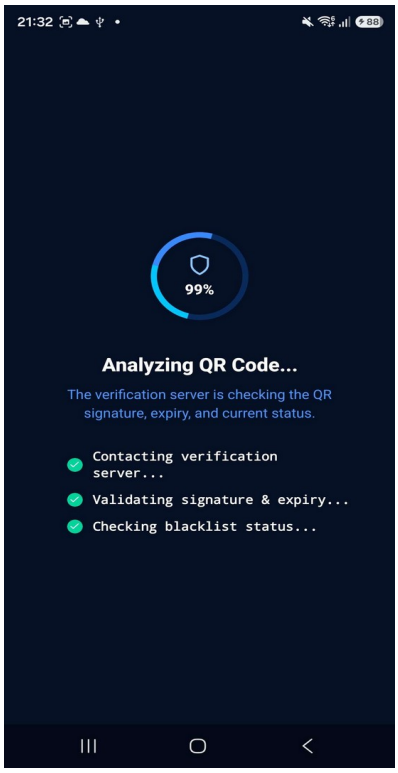
7.3.6 Mobile App User Dashboard and Profile



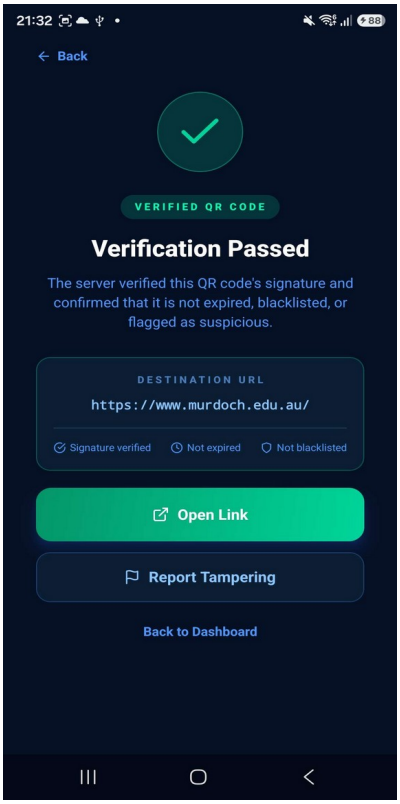


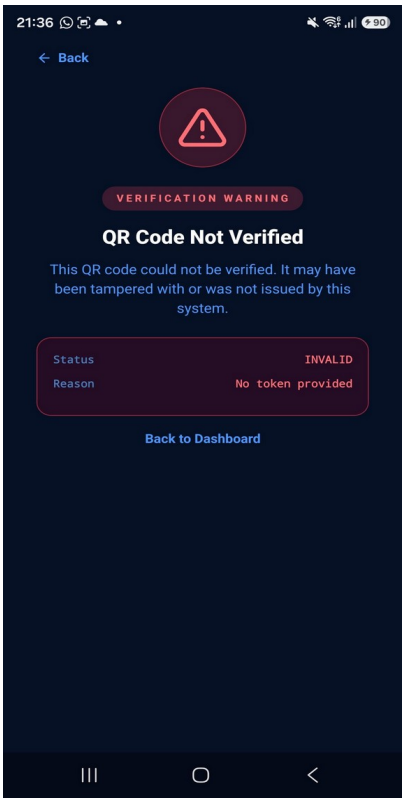
7.3.7 Mobile App QR Scan



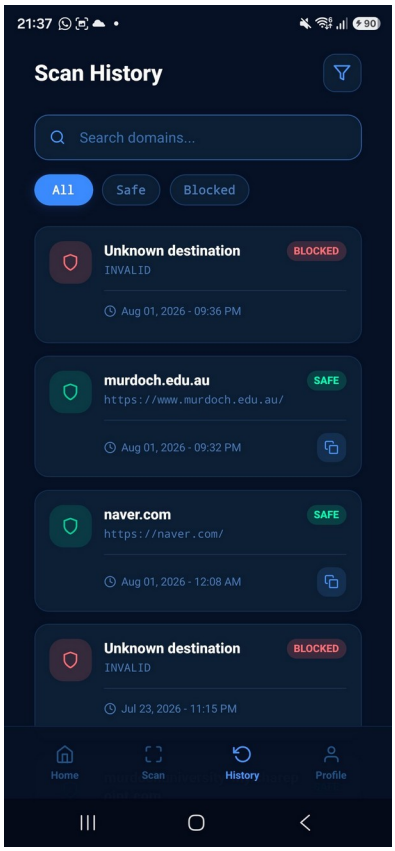


7.3.8 Mobile App QR Code Safe and Warning Result





7.3.9 Mobile App User History



7.4 Web Page Interfaces

The web page designs created in Figma will be implemented using React.

7.4.1 Admin Login Page

The admin login page provides secure access to the web-based management dashboard.

Design Components:

- Project branding with logo and name displayed prominently
- Email input field
- Password input field
- “Sign in” button as the primary action
- Error message display for invalid credentials

Figure 7.4.1.1 Register Page

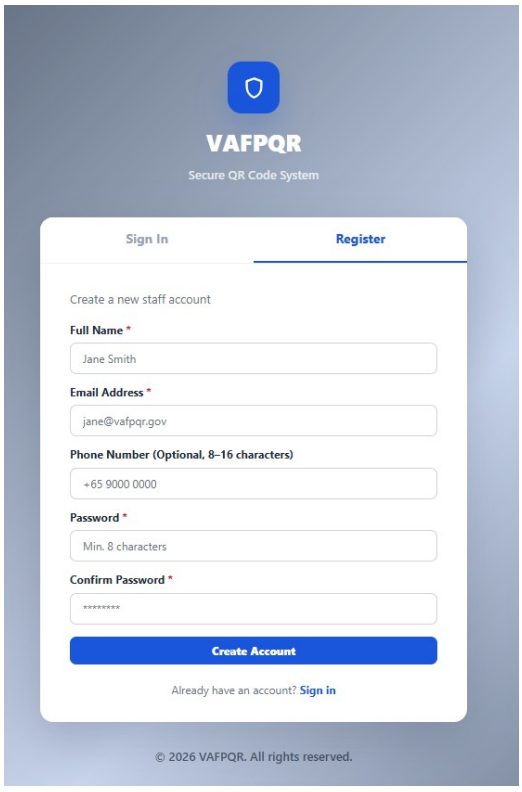
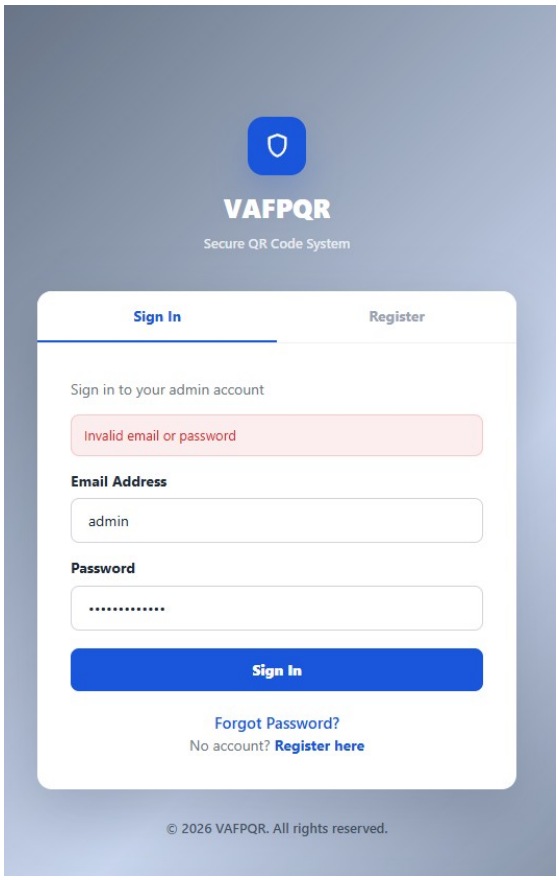


Figure 7.4.1.2 Sign in error page



7.4.2 QR Code Display Page (Authenticated Admin)

This authenticated page is displayed to administrators after generating a new QR code.
Page Components:

- QR Code Image: Prominently displayed generated QR code (PNG format)
- QR Code Details:
  - QR.id (UUID)
  - Destination URL
  - Expiry Date and Time
  - Current Status (Active, Suspicious, Expired, Blacklisted)
- Actions:
  - "Download PNG" button (PNG download)
  - "Print" button (browser print)
  - Metrics Summary: Scan count, alert count, and recent scan activity

Figure 7.4.2.1 Generated QR Code Page

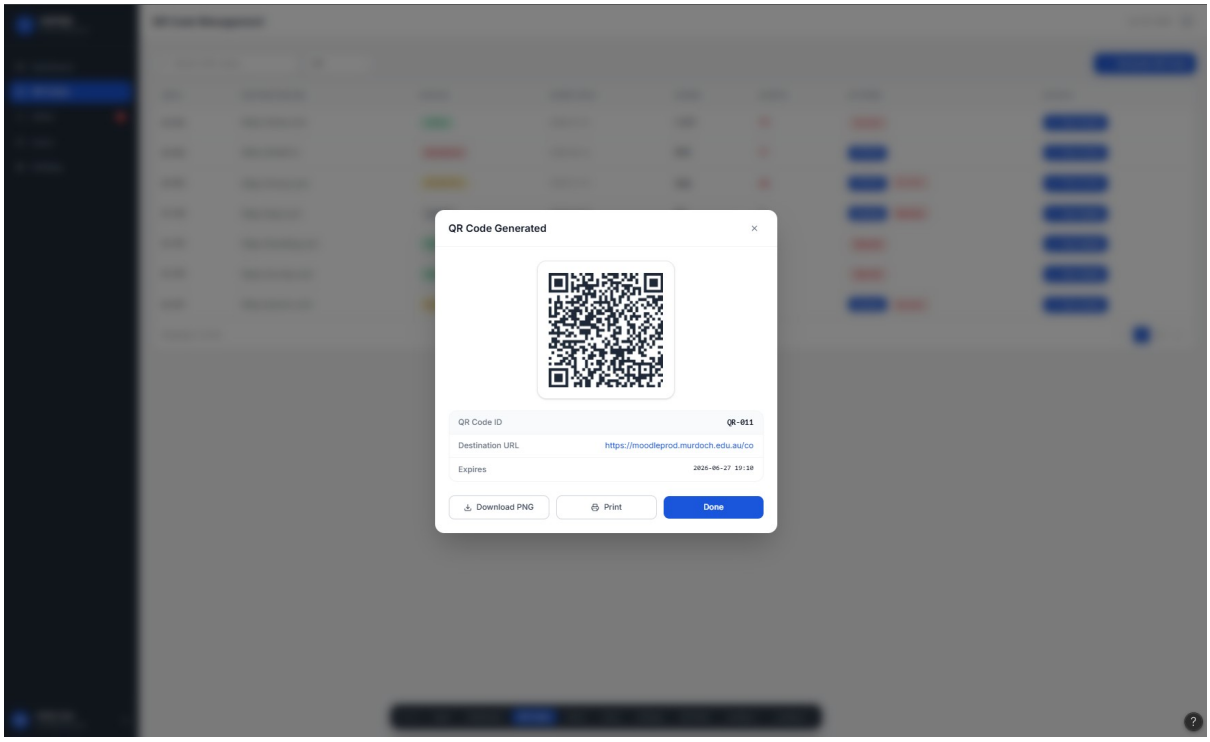


Figure 7.4.2.2 QR Code Print Page

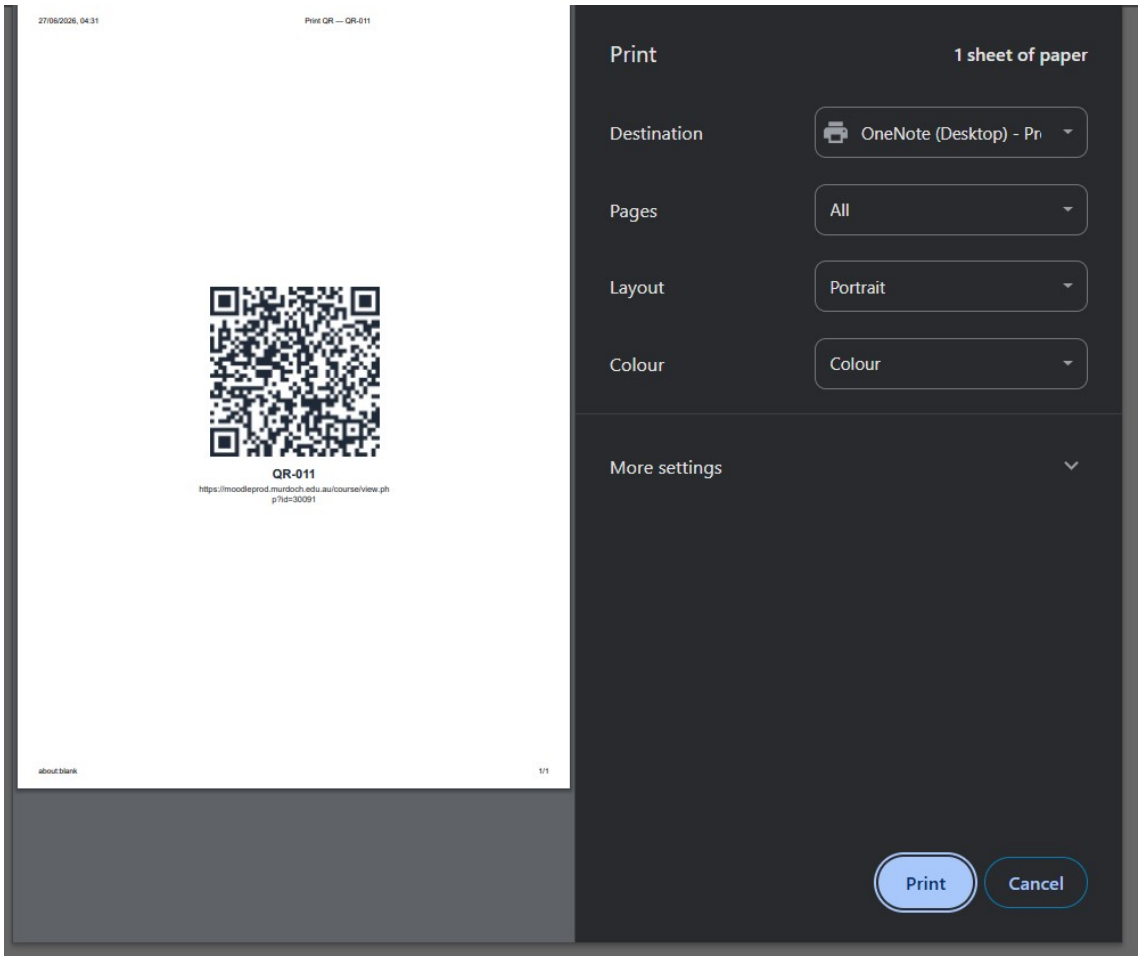


Figure 7.4.2.3 Generated QR Code Downloaded Page

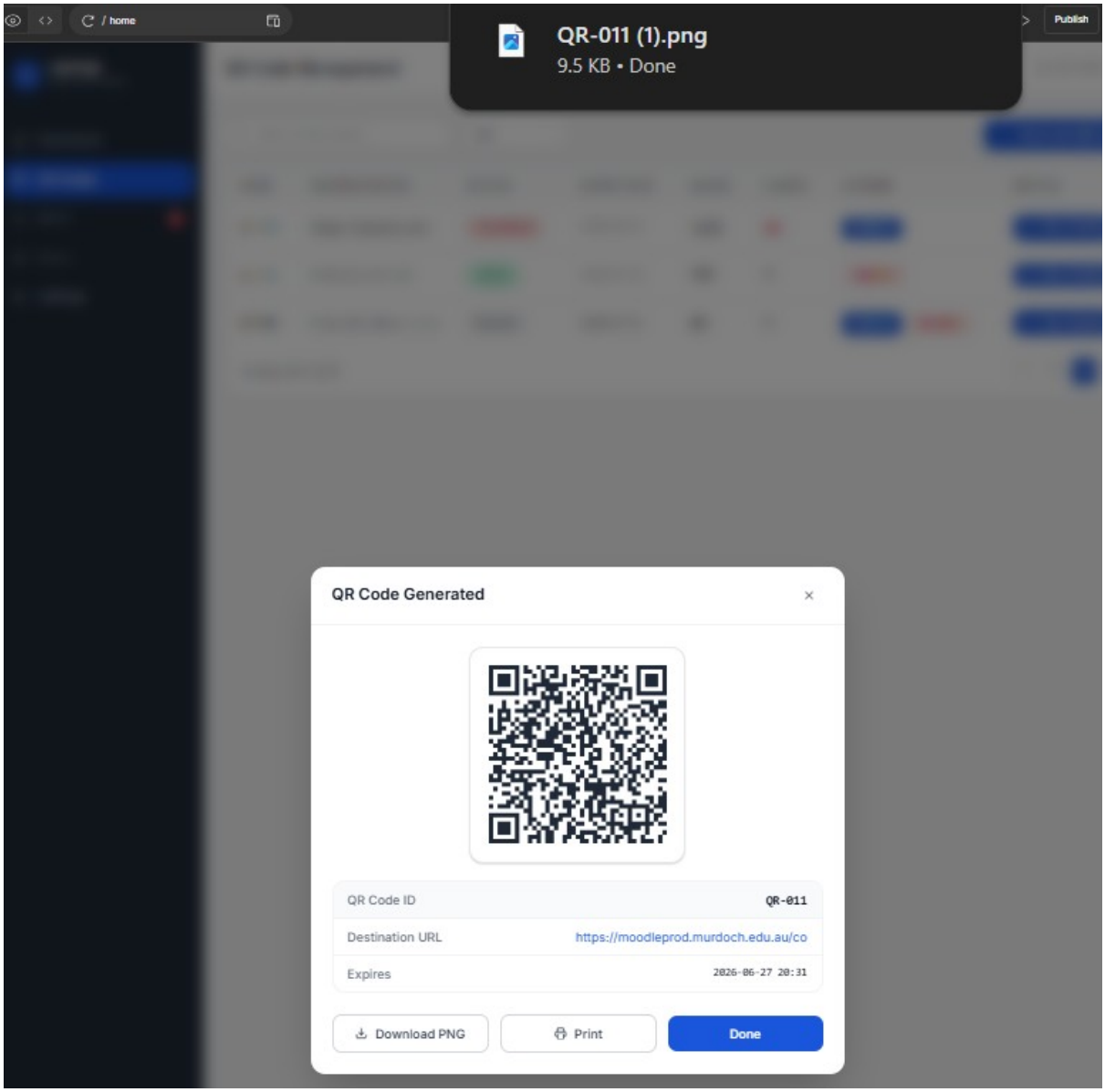
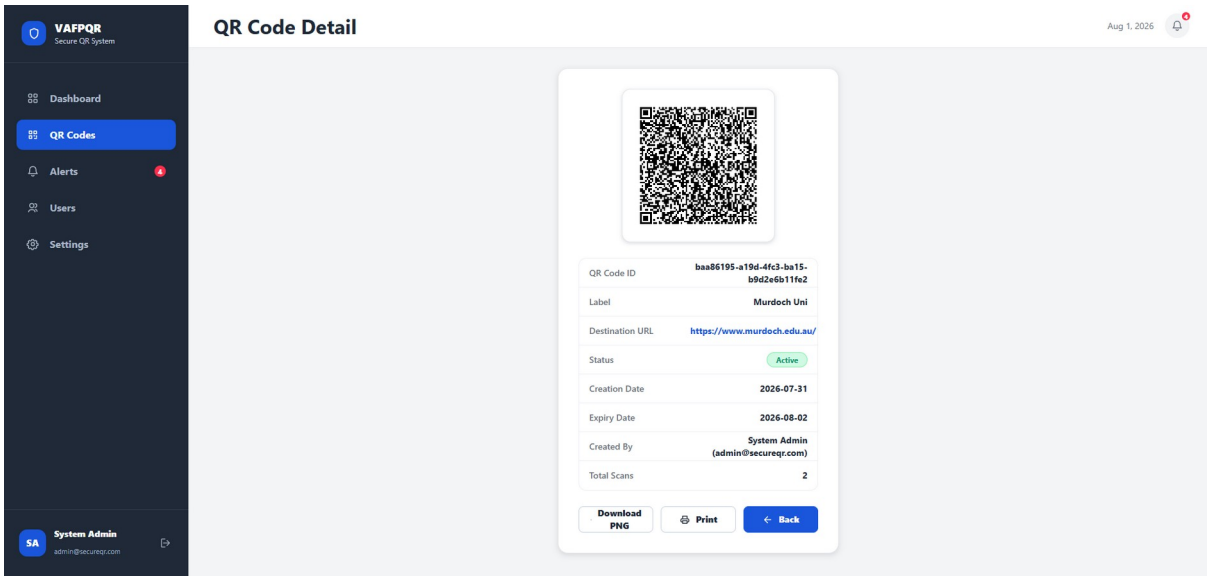


Figure 7.4.2.4 Generated QR Code Details Page



7.4.3 Admin Management Dashboard

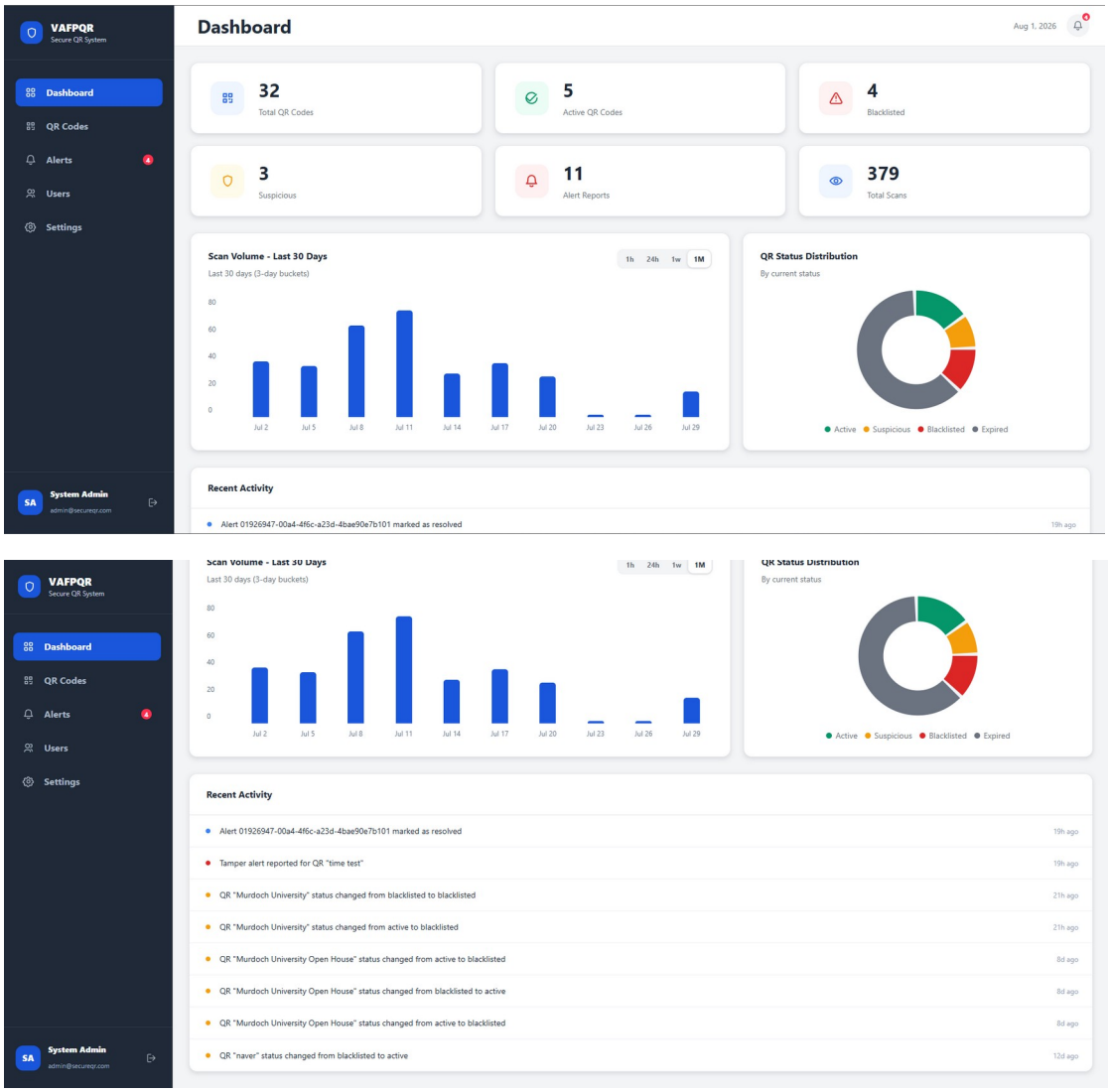
This secure page provides system administrators with full control over the QR code platform.

Quick Actions: Dashboard Page, QR Code page, Alert Page, Users Page, Settings Page

Dashboard Page (/dashboard):

- Metric Cards: Total QR codes, Active, Suspicious, Blacklisted, and Total Scans
- Scan Volume Chart: Bar chart showing scan activity over time (last 1 hour , last 24 hours, 1 week, 1 month)
- QR Status Distribution: Donut Chart for QR Status(Active, Suspicious, Blacklisted, Expired)
- Recent Activity: Table showing 8 most recent activities

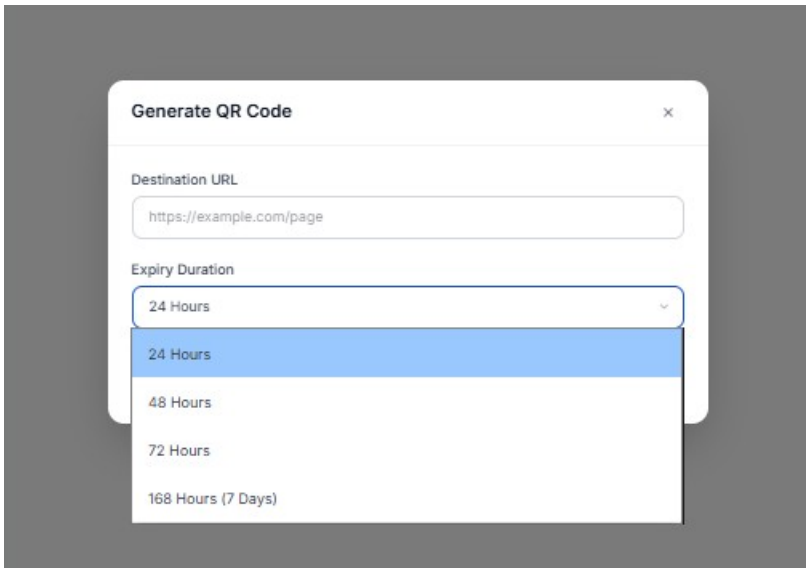
Figure 7.4.3.1 Dashboard Overview Page



QR Code Management Page (/qrcodes):

- Generate QR Code Form:
  - Destination URL input
  - Expiry Hours dropdown (24, 48,72,168)
  - "Generate QR Code" button
  - Result display with QR code image and details

Figure 7.4.3.2 QR Code Expiry Dropdown



- QR Codes Listing Table:
  - Columns: QR.id, Destination URL, Status (badge), Expiry Date, Alert Count, Actions, Details
  - Status Filter (All, Active, Suspicious, Expired, Blacklisted)
  - Search by QR.id or URL
  - Actions: Blacklist, Activate, View Details
- Pagination: 10, 25, 50 records per page

Figure 7.4.3.3 QR Codes Listing Table

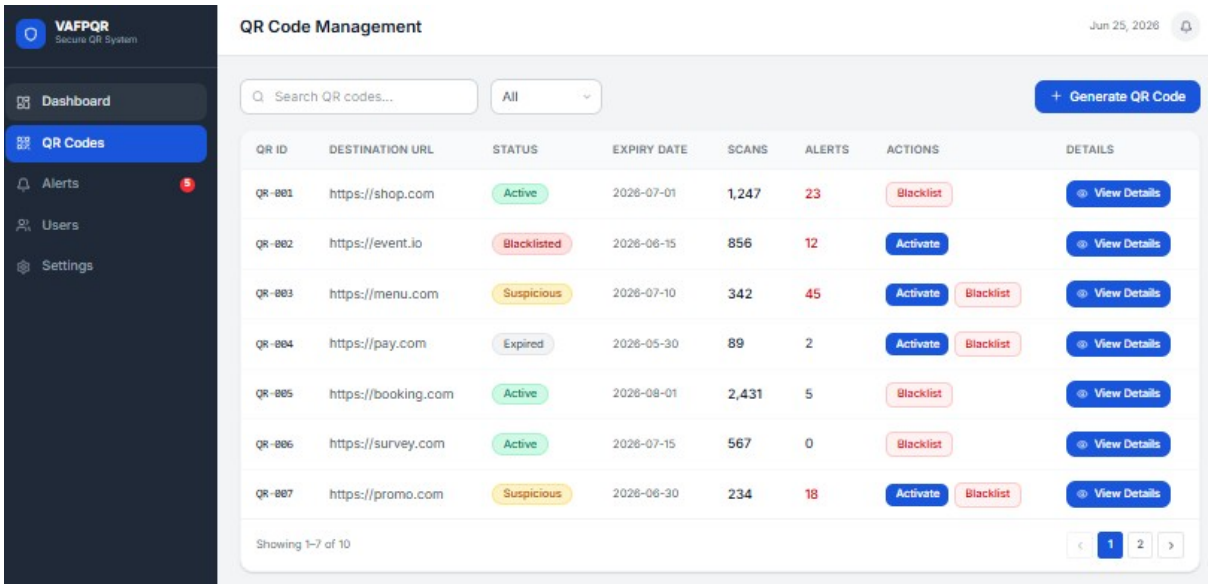


Figure 7.4.3.4 QR Status Dropdown

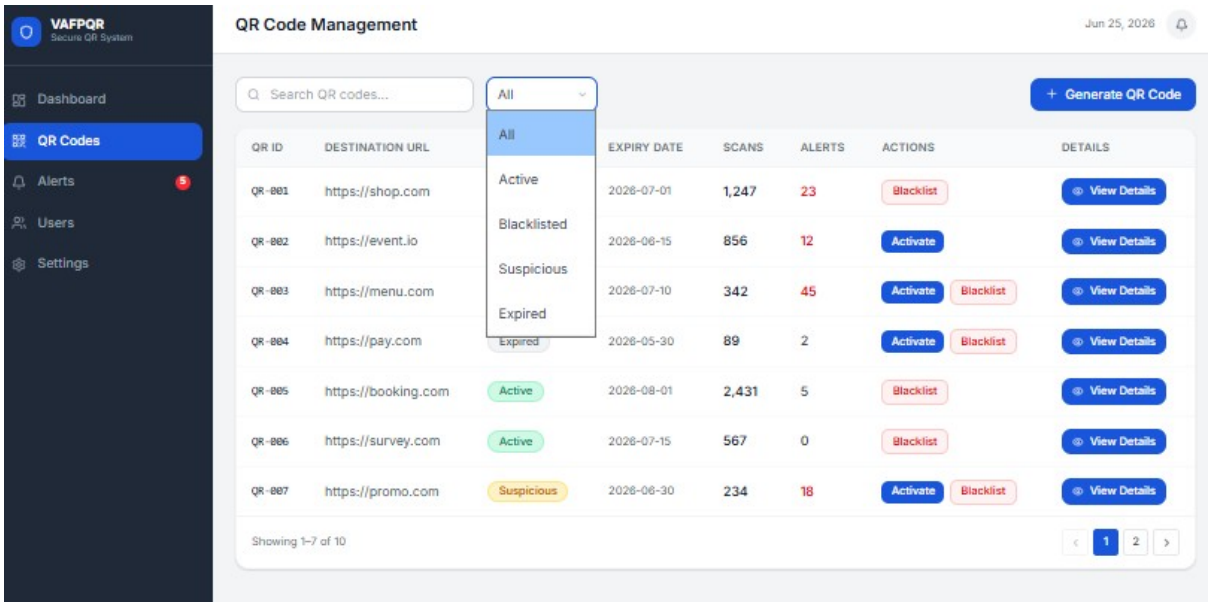
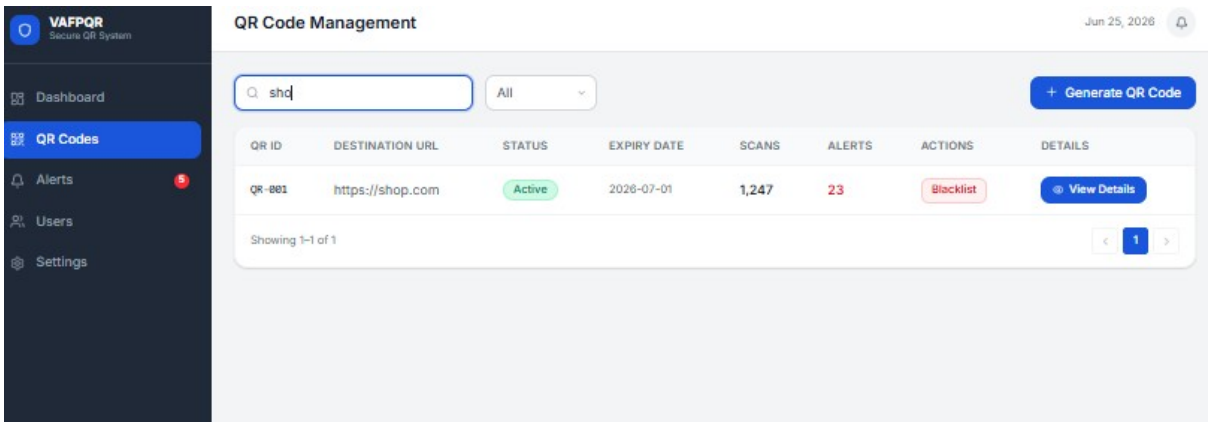


Figure 7.4.3.5 Search for Unique QR



Alerts Review Page (/alerts):

- Alerts Listing Table:
  - Columns: Alert ID, QR Code, Username (if provided), GPS Location, Description, Status, Submission Date and time, Details
  - Details column with expandable view
  - Status Filter (All, New, Reviewed, Resolved)
  - Actions: View Details, Mark as Reviewed, Mark as Resolved
- Alert Detail Modal:
  - Full-size photo evidence (Upload Photo, Take Photo)
  - QR Code ID
  - Full description
  - GPS location with map link
  - User details (Name, Preferred Contact)
  - Submission Date and Time
  - Admin notes
  -

Figure 7.4.3.6 Alert Page

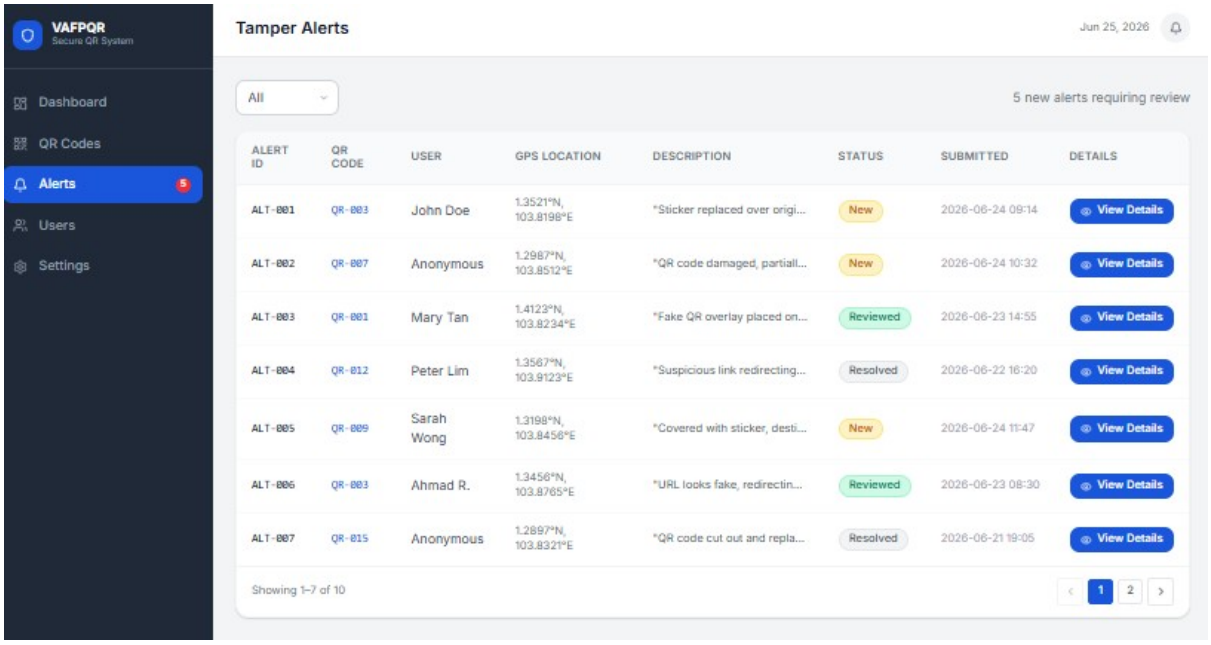
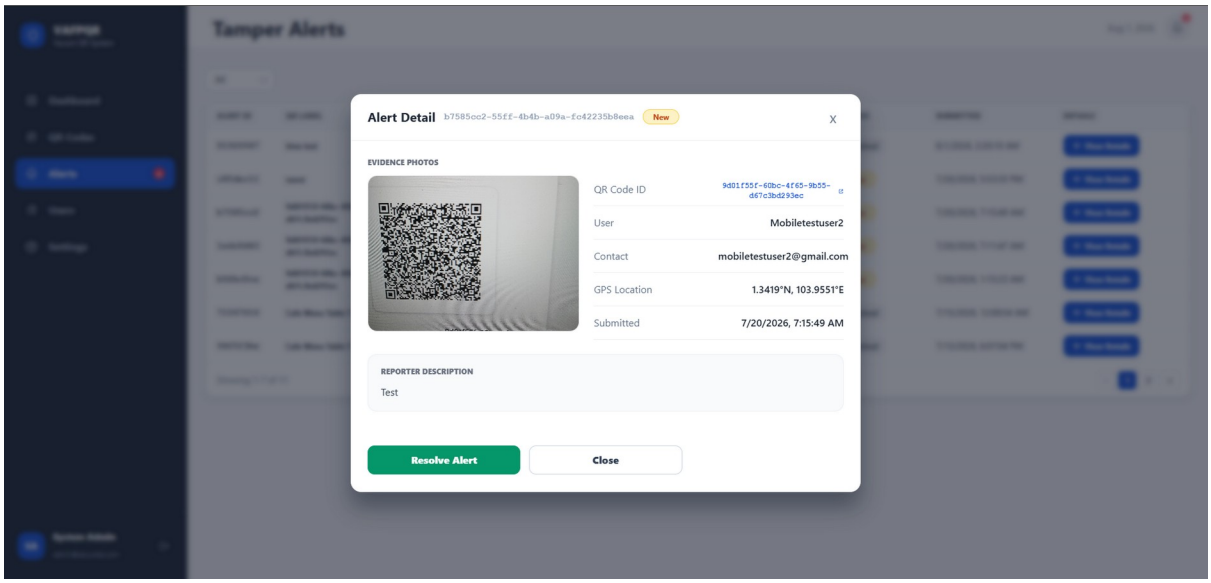


Figure 7.4.3.7 Alert View Details Page



7.4.4 Users Management Page

This secure page allows administrators to view and manage system users.

Design Components:

Metric Cards:

- Total Users card showing the total number of registered users
- Active card showing the number of active users
- Suspended card showing the number of suspended users
- 2FA Enabled card showing the number of users with two-factor authentication enabled

Users Table:

- Search bar to search for users by name or email
- Table columns:
    - User: Username and email address
    - Role: Super Admin, Admin, Reviewer, or User role badges
    - Last Login: Date and time of the user's last login
    - Actions: Suspend or View buttons based on selected User

Add User Button:

- Primary action button to add a new user
- Opens a modal or navigates to an add user form

Figure 7.4.4.1 User Page

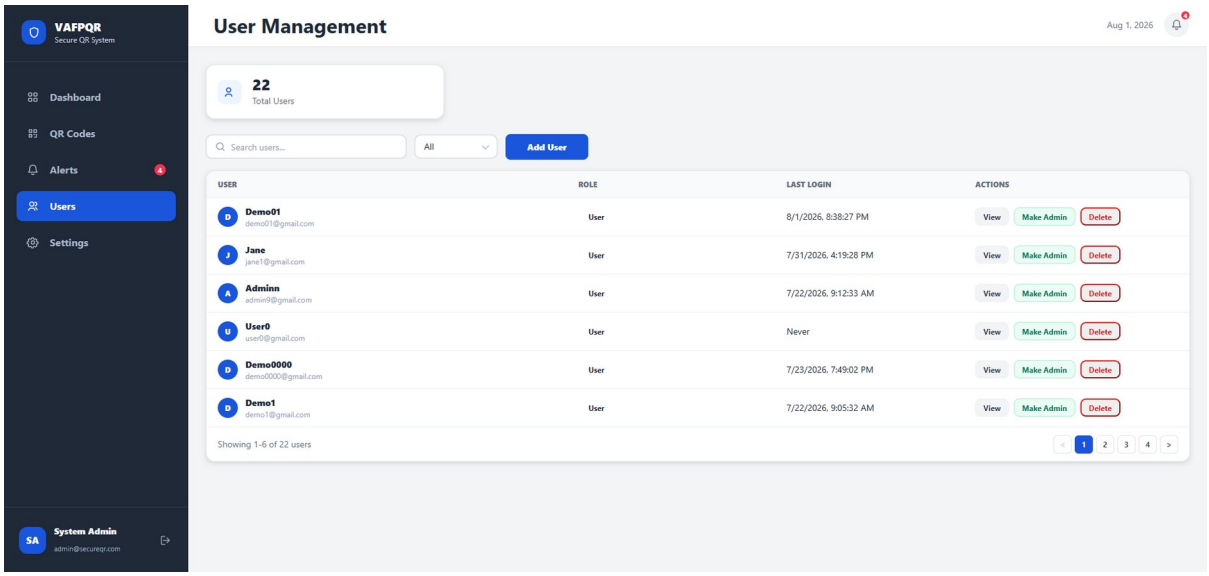


Figure 7.4.4.2 User Role Dropdown Page

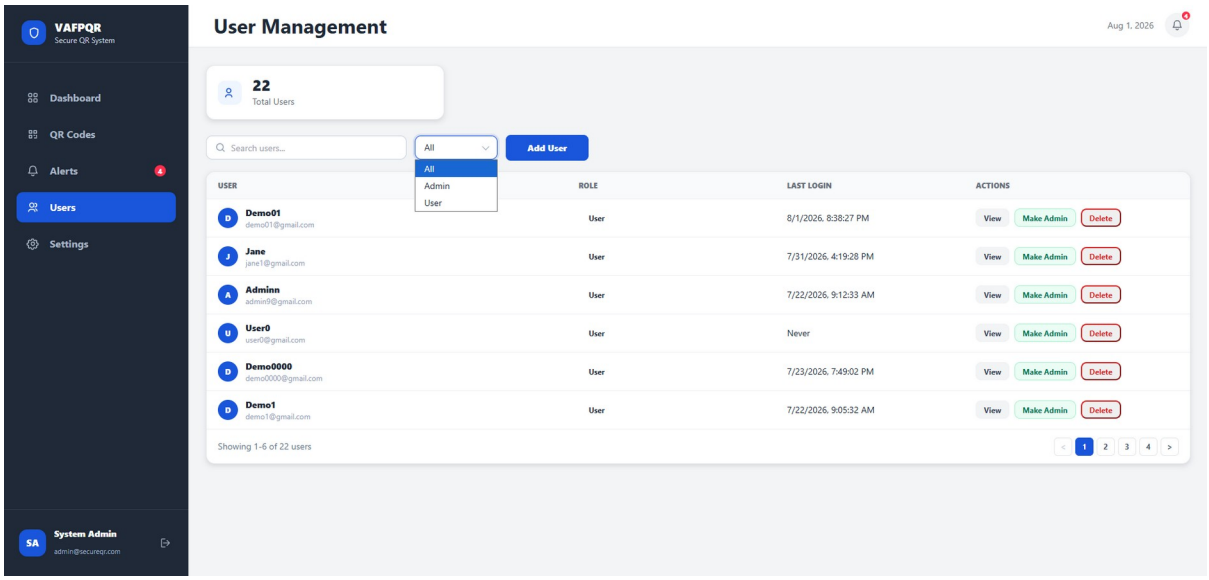
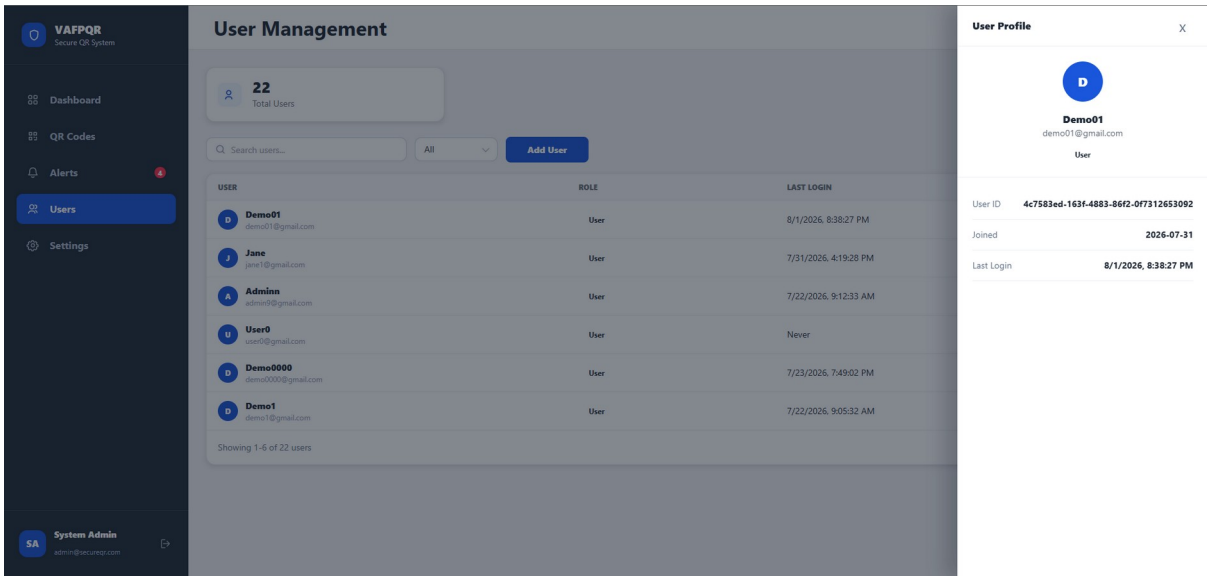


Figure 7.4.4.4 User Action View Page



7.4.5 Settings Page

This secure page allows administrators to view and manage their account settings.

Design Components:

Account Profile Section:

- User avatar with initials or profile picture
- Role badge showing user role
- Full Name field showing the user's full name
- Email Address field showing the user's email
- Phone Number field showing the user's contact number
- Password and Security section showing Change Password Function

Notifications Section:

- New Alert Reports toggle for Alert notifications

Figure 7.4.5.1 Setting Page

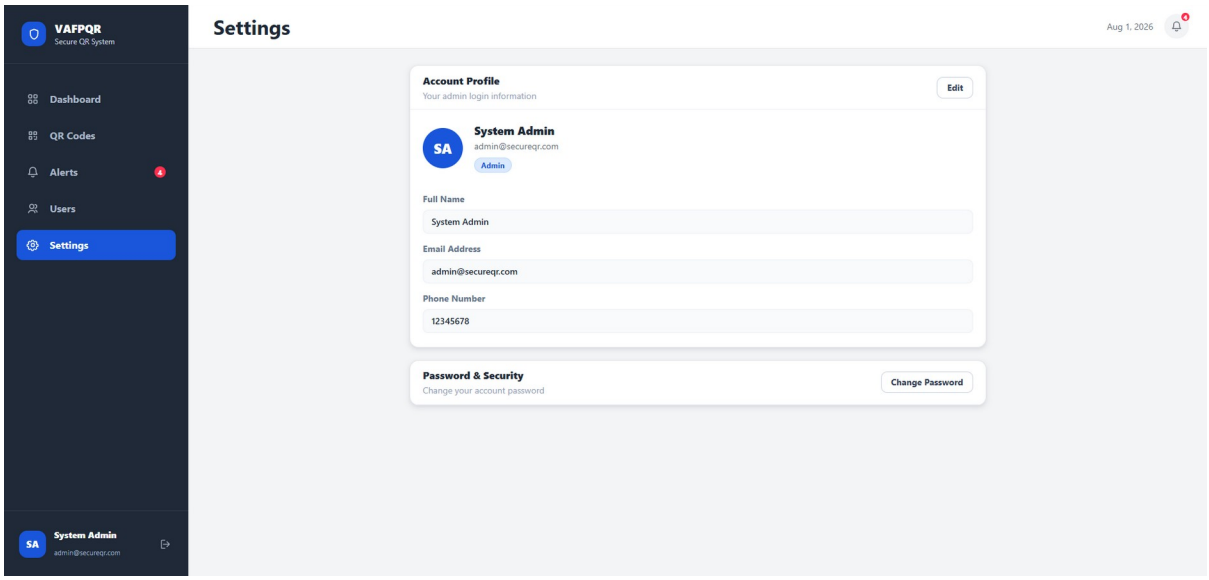
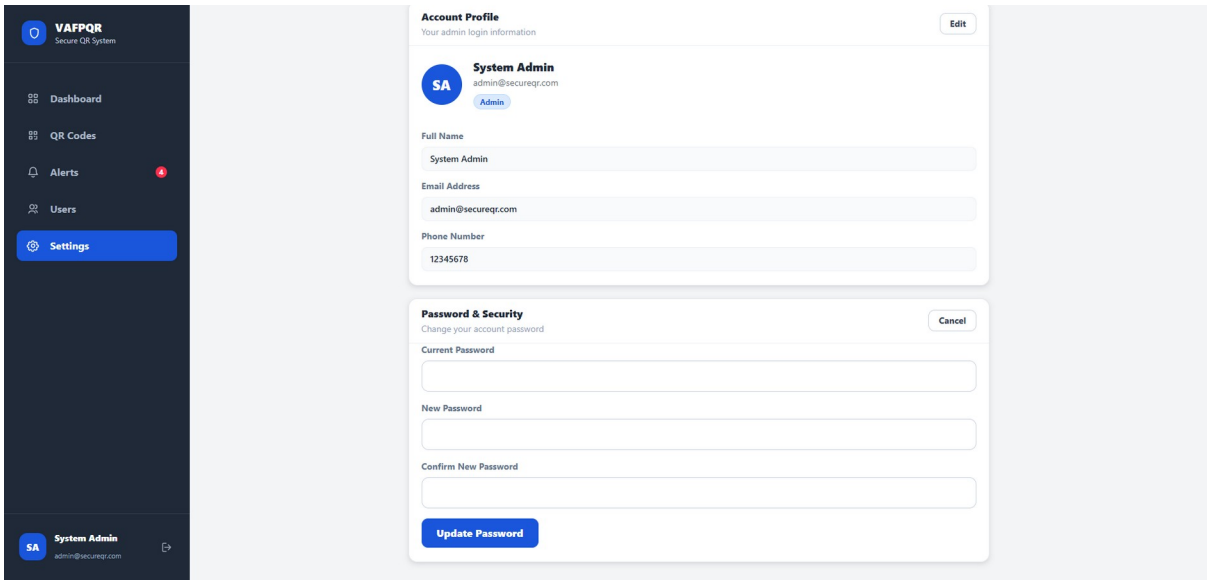


Figure 7.4.5.2 Change Password Page



7.5 GUI Components and Input and Output Specifications

Component Selection:

Component	Usage
Text Input	Login email, password, full name, destination URL, search, description, username, contact
Buttons	Sign in, Register, Generate, Download PNG, Print, Submit, Blacklist, Activate, Logout, Add User, Suspend, View, Export QR Code data
Toggle Switch	New Alert Reports
Dropdown	Status filtering, Role filtering, Expiry hours selection
File Upload	Upload Photo, Take Photo for evidence upload
Table	QR codes listing, Alerts listing, Users listing, Recent Activity
Charts	Scan Volume bar chart, QR Status Distribution donut chart
Cards	Metric summary cards
Dialog	Confirm dialogs, Alert detail modal, Add User modal
Badge	Status indicators for Active, Blacklisted, Suspended, Expired Role Indicators for Admin, User
Navigation Menu	Sidebar navigation with Dashboard, QR Codes, Alerts, Users, Settings

Input Specifications:

Component	Usage	Component	Usage
Text Input	Login email, password, full name, destination URL, search, description, user name, contact	Text Input	Login email, password, full name, destination URL, search, description, user name, contact
Buttons	Sign in, Register, Generate, Download PNG, Print, Submit, Blacklist, Activate, Logout, Add User, Suspend, View, Export QR Code data	Buttons	Sign in, Register, Generate, Download PNG, Print, Submit, Blacklist, Activate, Logout, Add User, Suspend, View, Export QR Code data
Toggle Switch	New Alert Reports	Toggle Switch	New Alert Reports

Dropdown	Status filtering, Expiry hours selection	Dropdown	Status filtering, Expiry hours selection
File Upload	Upload Photo, Take Photo for evidence upload	File Upload	Upload Photo, Take Photo for evidence upload
Table	QR codes listing, Alerts listing, Users listing, Recent Activity	Table	QR codes listing, Alerts listing, Users listing, Recent Activity
Charts	Scan Volume bar chart, QR Status Distribution donut chart	Charts	Scan Volume bar chart, QR Status Distribution donut chart
Cards	Metric summary cards	Cards	Metric summary cards
Dialog	Confirm dialogs, Alert detail modal, Add User modal	Dialog	Confirm dialogs, Alert detail modal, Add User modal
Badge	Status indicators for Active, Blacklisted, Suspended, Expired Role Indicators for Admin, User	Badge	Status indicators for Active, Blacklisted, Suspended, Expired Role Indicators for Admin, User
Navigation Menu	Sidebar navigation with Dashboard, QR Codes, Alerts, Users, Settings	Navigation Menu	Sidebar navigation with Dashboard, QR Codes, Alerts, Users, Settings

Output Specifications:

Output	Format	Purpose
QR Code Image	PNG as downloadable	Display, download, print
Validation Result	Colour coded status badge	User feedback on landing page
Scan Metrics	Bar chart, Donut chart	Admin monitoring
Tamper Report	Table with expandable details	Admin review
Dashboard Summary	Cards with counts	System overview
User List	Table with user details	Admin user management
Recent Activity	Table with recent actions	System activity monitoring

Error Handling:

Error Scenario	User Feedback
Invalid login credentials	Invalid email or password message
Email already registered	This email is already registered message
Password too weak	Password must be at least 8 characters message
Passwords do not match	Passwords do not match message
Invalid URL format	Red inline validation message under field
Expiry hours not selected	Please select expiry duration message
QR code generation fails	Failed to generate QR code message
Blacklist action fails	Error dialog with retry option
Empty description on report	Description is required inline message
Network timeout	Connection lost. Please try again message

8. Appendices

Appendix A: Deliverable Task Breakdown Statement: completed, signed, scanned and copied into document

Deliverable Name:

Project Team Name	Team Number
VAFPQR	FT02

Name	Student ID	Deliverable Percentage Completed	Signature	Date
Patricia Mikaela Marie Jusay Orenza	[redacted]	16.66%		1st August 2026
Lim Yu Jun Felixia	[redacted]	16.66%		1st August 2026
Roy Yoo Hojin	[redacted]	16.66%		1st August 2026
Gu QiYu	[redacted]	16.66%		1st August 2026
Alvin Rui Long Yeow	[redacted]	16.66%		1st August 2026
Vanessa Tan	[redacted]	16.66%		1st August 2026

Appendix B: Glossary/data dictionary

The data dictionary that contains all the details about the VAFPQR Secure QR Code Authentication System is presented in Section 4.3 Data Dictionary (page 12-17).

This appendix will serve as an initial point of reference to all the data definitions, which include:

- Descriptions of Entities such as User, QR Code, QR Scan Log, and QR Incident Report
- Specifications of Attributes, which include data types, keys, and constraints
- Primary and Foreign Key Relationship
- Domain Integrity Rules and Constraints
- Constraints on the business data

For more information, you can refer to Section 4.3.

Appendix C: Glossary of terms and definitions and acronyms

Term/ Abbreviation	Definition
Admin User	An authenticated system administrator who accesses the web dashboard to manage QR codes and review tamper alerts.
App User	A member of the public who has installed the Secure QR mobile application and uses it to scan and authenticate QR codes.
API	Application Programming Interface. The set of HTTP endpoints exposed by the backend server.
APK	Android Package Kit. The file format used to distribute and install Android applications.
bcrypt	A cryptographic password hashing function. Used to hash admin passwords before storage.
CRUD	Create, Read, Update, Delete. The four fundamental operations performed on data.
DFD	Data Flow Diagram. A graphical representation of data movement through a system.
ERD	Entity-Relationship Diagram. A diagram showing the entities in a database and their relationships.
Express	A minimal and flexible Node.js web application framework used to build the backend API server.
GPS	Global Positioning System. Used to capture geographic coordinates from the mobile device.
HCI	Human-Computer Interaction. The design of interfaces through which humans interact with software systems.
HMAC	Hash-based Message Authentication Code. Used internally by JWT HS256 to sign tokens.
HTTPS	Hypertext Transfer Protocol Secure. HTTP

	encrypted with TLS. Required for all production API communication.
ISO/IEC 18004	The international standard defining the QR code specification.
JWT	JSON Web Token. A compact, self-contained token used to represent claims. Used here to sign and verify QR code authenticity.
Multer	A Node.js middleware for handling multipart/form-data, used for photo uploads in tamper alert reports.
Node.js	A JavaScript runtime built on Chrome's V8 engine. Used as the backend server runtime.
ORM	Object-Relational Mapping. A programming technique that maps database tables to application objects. Prisma is the ORM used in this project.
PostgreSQL	An open-source relational database management system. Used as the data persistence layer.
Prisma	A next-generation ORM for Node.js and TypeScript. Used for database schema management and type-safe queries.
QR Code	Quick Response Code. A two-dimensional barcode capable of encoding text, URLs, or other data.
React	A JavaScript library for building user interfaces. Used for the three web pages.
React Native	A framework for building native mobile applications using React. Used for the Android mobile application.
Regular User	A member of the public who scans a QR code using a standard device camera, without the dedicated app.
REST	Representational State Transfer. An architectural style for designing networked APIs using HTTP.

TLS	Transport Layer Security. The cryptographic protocol providing HTTPS encryption.
UUID	Universally Unique Identifier. A 128-bit identifier used as the primary key for all database entities.
User-Agent	An HTTP request header identifying the client software. The mobile app sends 'SecureQRApp/1.0' to distinguish itself from browser requests.
VAFPQR	The project team name for the Secure QR Code Development group.